

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты гр. 4344,
4341

Кочененко А.А.
Благодарная К.О.
Палаев И.С.
Стиврин А.Д.

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Стыврин А.Д.

Группа: 4344

Тема практики: UI-тестирование сайта Demoblaze

Задание на практику:

Разработка и внедрение программного решения для автоматизированного UI-тестирования отдельных функциональных модулей веб-приложения с применением современного набора инструментов. В рамках практики необходимо сформировать детальный чек-лист, состоящий из 10 тестовых сценариев с суммарным количеством не менее 50 пользовательских действий, а также выполнить их программную реализацию на языке Java с использованием Selenide и JUnit 5. Архитектура проекта должна быть организована на основе паттерна Page Object Model и компонентного подхода, что позволяет разделить логику тестовых сценариев, описание страниц и работу с элементами интерфейса, упрощая поддержку и развитие кода. Также работа предполагает настройку системы логирования Log4j2 для отслеживания выполнения тестов, ведение совместной разработки через репозиторий GitHub и подготовку итогового отчета с описанием структуры классов, методов и соответствующих UML-диаграмм.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студенты

Руководитель

Кочененко А.А.
Благодарная К.О.
Палаев И.С.
Стыврин А.Д.

А.М. Шевелёва

АННОТАЦИЯ

В отчёте рассматривается разработка и реализация автоматизированного UI-тестирования для демонстрационного интернет-магазина Demoblaze. В процессе работы были изучены основные возможности сайта и подготовлен чек-лист пользовательских сценариев, отражающих ключевые действия покупателя: переход по категориям товаров, открытие карточки продукта, добавление товаров в корзину, удаление позиций, проверка состояния корзины, оформление заказа и отмена оформления.

Для реализации тестов был использован язык программирования Java. Структура проекта построена с применением паттерна Page Object Model, благодаря чему тестовые сценарии отделены от логики взаимодействия с элементами веб-интерфейса. Такой подход позволил сделать код более понятным, упорядоченным и удобным для дальнейшего сопровождения.

Автоматизированные проверки выполнены с использованием фреймворка Selenide, библиотеки JUnit 5, системы сборки Maven и средств логирования. В рамках проекта были описаны классы страниц, элементы интерфейса и тестовые классы, а также подготовлена сопроводительная документация с описанием реализованных тестов, методов и UML-диаграмм. Итоговая версия проекта размещена в репозитории GitHub, где отражена история изменений и вклад участников команды.

SUMMARY

This report focuses on the development and implementation of automated UI testing for the demonstration online store Demoblaze. During the work, the main features of the website were studied, and a checklist of user scenarios was prepared. These scenarios reflect the key actions of a customer: navigating product categories, opening a product card, adding products to the cart, removing items, checking the cart state, placing an order, and canceling the checkout process.

The tests were implemented using the Java programming language. The project structure is based on the Page Object Model pattern, which separates test scenarios from the logic of interacting with web interface elements. This approach made the code more understandable, organized, and convenient for further maintenance.

The automated checks were performed using the Selenide framework, the JUnit 5 library, the Maven build system, and logging tools. As part of the project, page classes, interface element classes, and test classes were described. Supporting documentation was also prepared, including descriptions of the implemented tests, methods, and UML class diagrams. The final version of the project was published in a GitHub repository, where the change history and the contribution of each team member are reflected.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	9
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	32
2.1. Описание программных классов и их методов	32
2.2. UML-диаграмма классов	35
2.3. Реализация взаимодействия компонентов.....	36
3 ИНДИВИДУАЛЬНАЯ РАБОТА	37
4 ОТЗЫВ РУКОВОДИТЕЛЯ	39
ЗАКЛЮЧЕНИЕ	40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42

ВВЕДЕНИЕ

Предметной областью данной практики является автоматизированное UI-тестирование пользовательских сценариев веб-приложения. В качестве тестируемой системы был выбран демонстрационный интернет-магазин Demoblaze [4], позволяющий воспроизвести основные действия пользователя при покупке товара: переход по категориям, просмотр карточки продукта, добавление товаров в корзину, удаление товаров, оформление заказа и отмену оформления. Автоматизация таких сценариев позволяет сократить объем ручной проверки, снизить вероятность ошибок и убедиться в корректной работе ключевых элементов интерфейса при повторном запуске тестов.

Целью практики является проектирование и программная реализация набора автоматизированных UI-тестов для проверки основного пользовательского пути в интернет-магазине Demoblaze. Основной упор делается на построение понятной и поддерживаемой структуры проекта с использованием объектно-ориентированного подхода и паттерна Page Object Model, при котором тестовые классы отделены от классов страниц и элементов интерфейса. В проекте реализованы отдельные классы страниц MainPage, ProductPage и CartPage, отвечающие за взаимодействие с главной страницей, карточкой товара и корзиной соответственно.

Для достижения поставленной цели необходимо решить следующие задачи:

Провести анализ функциональных возможностей сайта Demoblaze и сформировать чек-лист тестовых сценариев, покрывающих основные действия пользователя при выборе товара, работе с корзиной и оформлении заказа.

Разработать архитектуру проекта на языке Java с применением паттерна Page Object Model [1] и компонентного подхода, выделив базовые классы страниц и элементов интерфейса. В проекте для этого используются классы BasePage, BaseElement, Button и Input, позволяющие отдельно описывать страницы, кнопки и поля ввода.

Реализовать автоматизированные тесты с использованием Selenide [2] и JUnit 5 [3], обеспечив проверку перехода по категориям, открытия карточки товара, добавления и удаления товаров из корзины, отображения пустой корзины, открытия формы заказа, заполнения формы корректными данными и успешного оформления покупки.

Вынести повторяющиеся тестовые данные, такие как названия категорий, товаров и цена товара, в отдельный класс Constants, чтобы упростить поддержку тестов и избежать дублирования строковых значений в коде.

Настроить базовое логирование выполнения тестов и действий с элементами интерфейса с помощью класса LoggerUtil, а также подготовить итоговую документацию с описанием реализованных тестов, классов, методов и UML-диаграмм проекта.

Вклад каждого участника:

Палаев И.С.: Архитектура и базовая настройка проекта: настройка pom.xml, подключение необходимых зависимостей, создание базовых классов для работы с элементами интерфейса BaseElement, Button, Input. Реализация класса BasePage с общими методами для страниц, а также класса LoggerUtil для логирования выполнения тестов и действий с элементами. Приведение работы с кнопками и полями ввода к единому виду, чтобы одни и те же действия можно было использовать в разных тестах и страницах.

Стиврин А.Д.: Реализация Page Objects для сайта Demoblaze: создание классов MainPage, ProductPage и CartPage, отвечающих за работу с главной страницей, карточкой товара и корзиной. Добавление методов для перехода по категориям, открытия товара, перехода в корзину, добавления товара, проверки данных на странице товара, удаления товара из корзины, открытия формы заказа и заполнения полей оформления покупки. Вынес основные тестовые данные, такие как названия товаров, категории и цена, в класс Constants, что упростило использование одинаковых значений в разных тестах.

Благодарная К.О.: создание чек-листа UI-тестирования для сайта Demoblaze, настройка базового тестового класса BaseTest, отвечающего за запуск браузера, открытие главной страницы сайта, установку размера окна, таймаута ожидания и закрытие браузера после выполнения теста. Тесты №1–4: проверка перехода по категориям Phones и Laptops, открытие карточки товара Samsung Galaxy S6, переход в корзину через верхнее меню, проверка добавления товара в корзину, включая появление сообщения Product added и отображение добавленного товара в таблице корзины.

Кочененко А.А.: Создание чек-листа UI-тестирования для сайта Demoblaze. Реализация тестов №5–9: добавление нескольких товаров в корзину, удаление товара из корзины, открытие формы оформления заказа, оформление заказа с корректными данными, отмена оформления заказа, проверка пустой корзины и появление товара после добавления в пустую корзину. Добавление assert-проверок для подтверждения наличия и отсутствия товаров в корзине, отображения формы заказа и сообщения об успешной покупке. Проверка соответствия реализованных тестов подготовленному чек-листу.

1 ПОСТАНОВКА ЗАДАЧИ

В итоговом проекте имеются 9 тестов:

1. Проверка перехода по категориям товаров Phones и Laptops.
2. Проверка открытия страницы товара Samsung Galaxy S6.
3. Проверка перехода в корзину через верхнее меню.
4. Проверка добавления товара в корзину.
5. Проверка добавления нескольких товаров в корзину.
6. Проверка удаления товара из корзины.
7. Проверка оформления заказа.
8. Проверка отмены оформления заказа.
9. Проверка отображения пустой корзины и добавления товара в пустую корзину.

Тест №1: Проверка перехода по категориям товаров

Цель: проверить корректность перехода по категориям товаров на главной странице сайта Demoblaze.

Ожидаемый результат: при выборе категории Phones открывается список товаров данной категории, в котором отображается товар Samsung Galaxy S6. При выборе категории Laptops открывается список товаров категории ноутбуков, в котором отображается товар Sony vaio i5.

В ходе выполнения теста пользователь открывает главную страницу сайта Demoblaze и поочередно выбирает категории Phones и Laptops. После перехода в каждую категорию проверяется, что в списке товаров отображается ожидаемый товар. Для категории Phones ожидаемым товаром является Samsung Galaxy S6, для категории Laptops — Sony vaio i5. Тест позволяет убедиться, что навигация по категориям работает корректно, а товары отображаются в соответствующих разделах.

Результаты действий представлены на рисунках (см. рисунки 1 – 3).

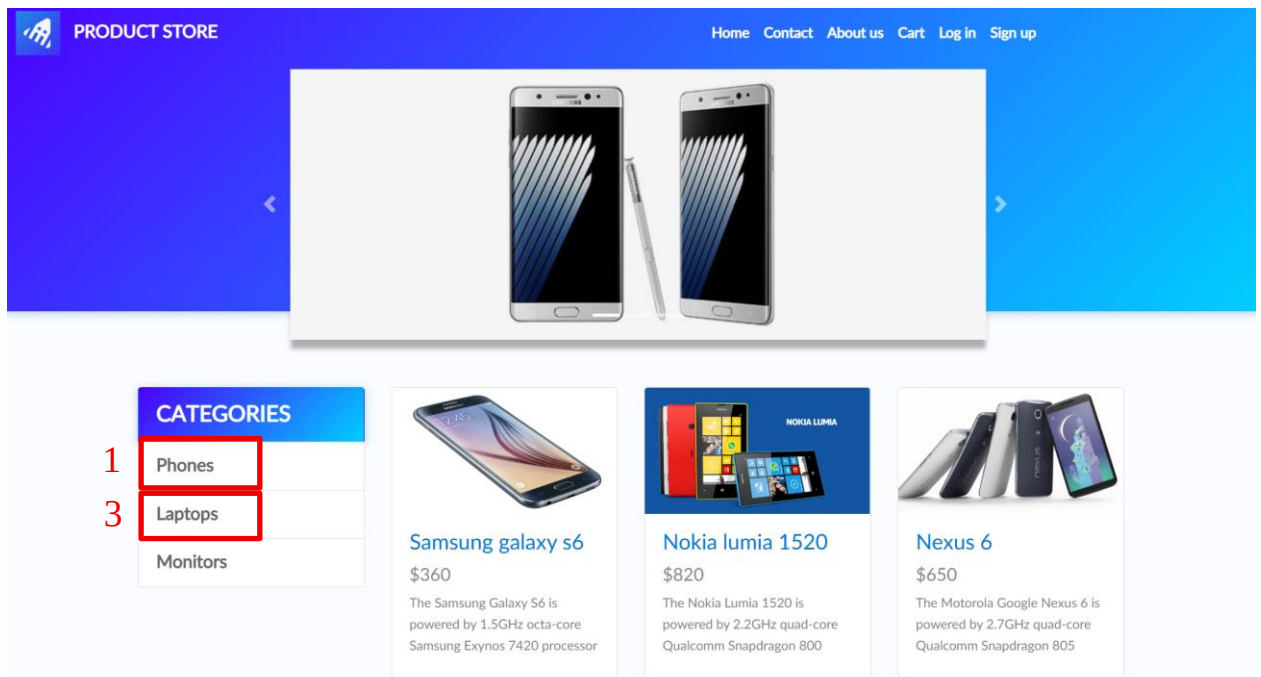


Рисунок 1 - Переход по категориям товаров. Часть 1

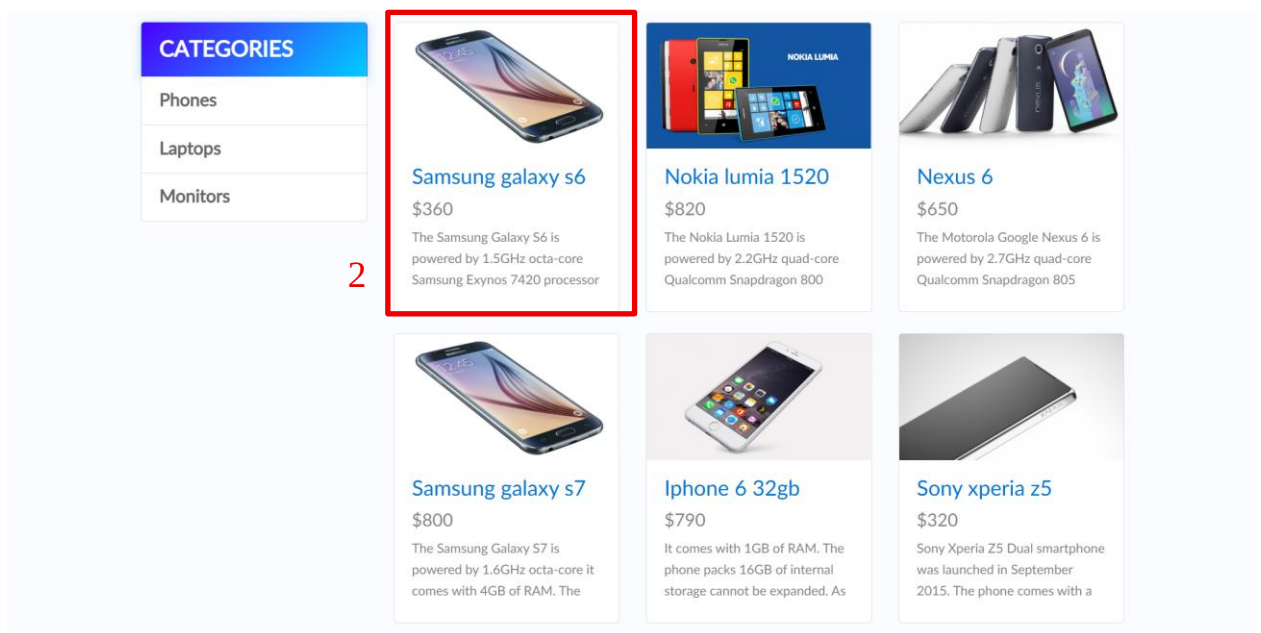


Рисунок 2 - Отображение товаров категории Phones

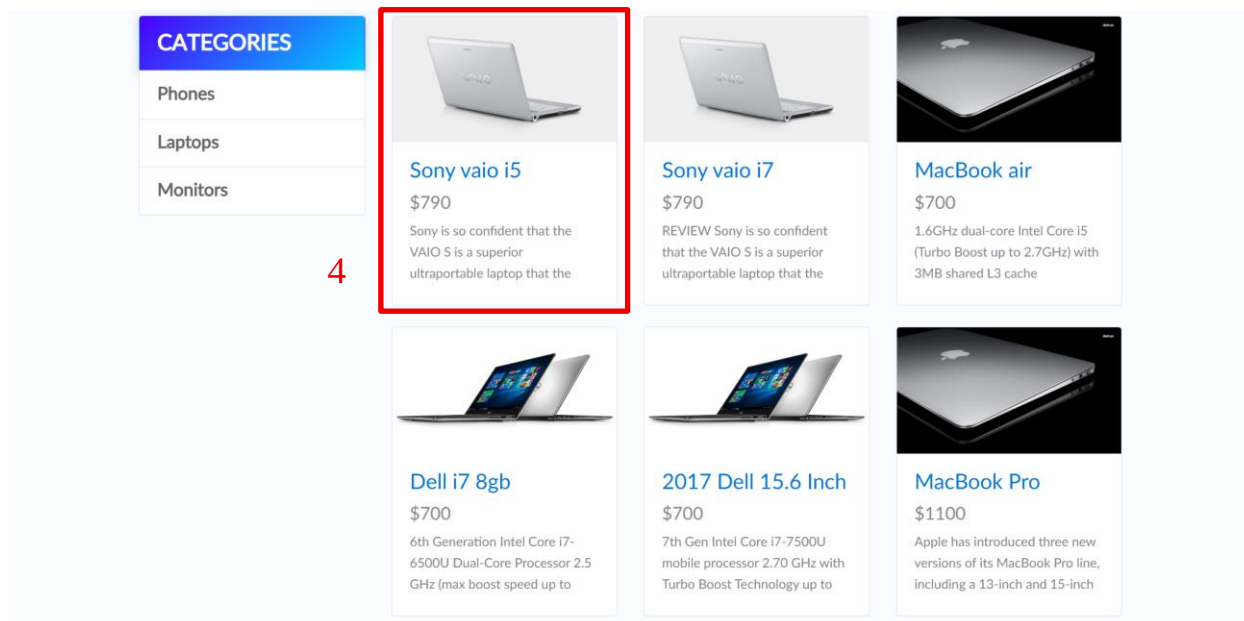


Рисунок 3 - Отображение товаров категории Laptops

Тест №2: Проверка открытия страницы товара

Цель: проверить корректность открытия карточки товара на сайте Demoblaze.

Ожидаемый результат: после выбора товара Samsung Galaxy S6 открывается его карточка. На странице товара отображаются название Samsung Galaxy S6, цена \$360, описание товара и кнопка Add to cart.

В ходе выполнения теста пользователь открывает главную страницу сайта Demoblaze, переходит в категорию Phones и выбирает товар Samsung Galaxy S6 из списка товаров. После перехода на страницу товара проверяется, что карточка открыта корректно и содержит все основные элементы, необходимые для дальнейшего добавления товара в корзину: название, цену, описание и кнопку Add to cart.

Результаты действий представлены на рисунках (см. рисунки 4 – 6).

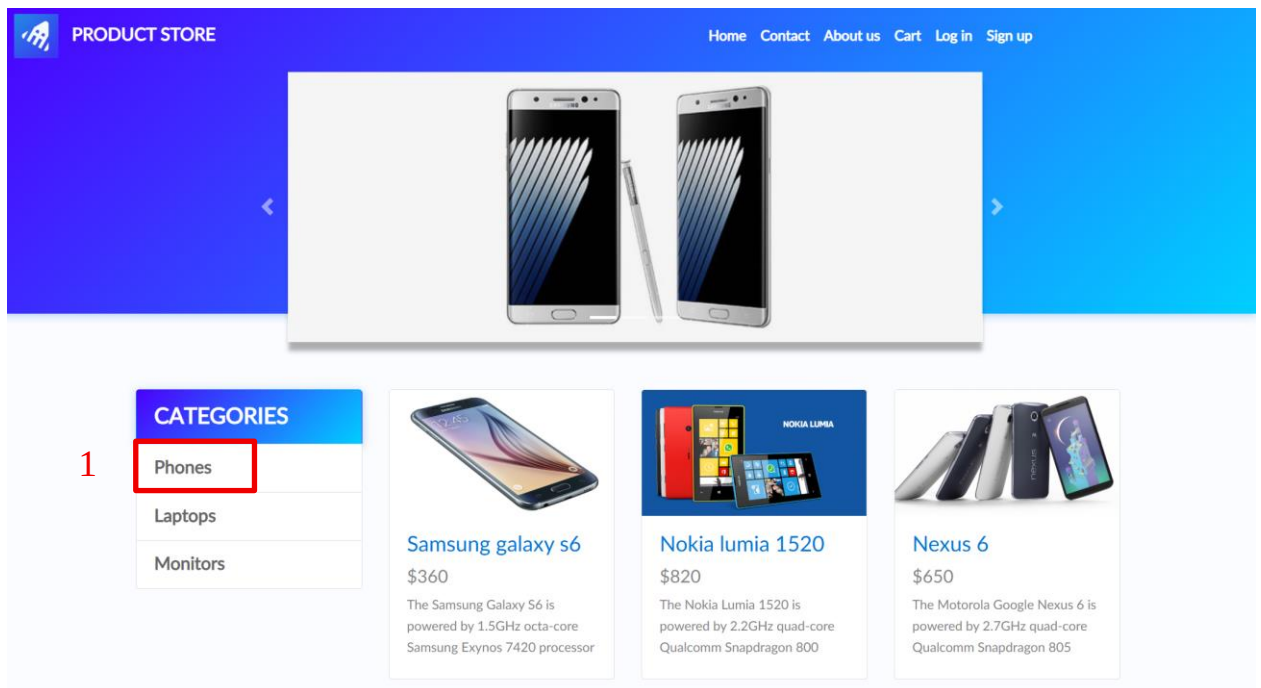


Рисунок 4 – Открытие страницы товара. Часть 1

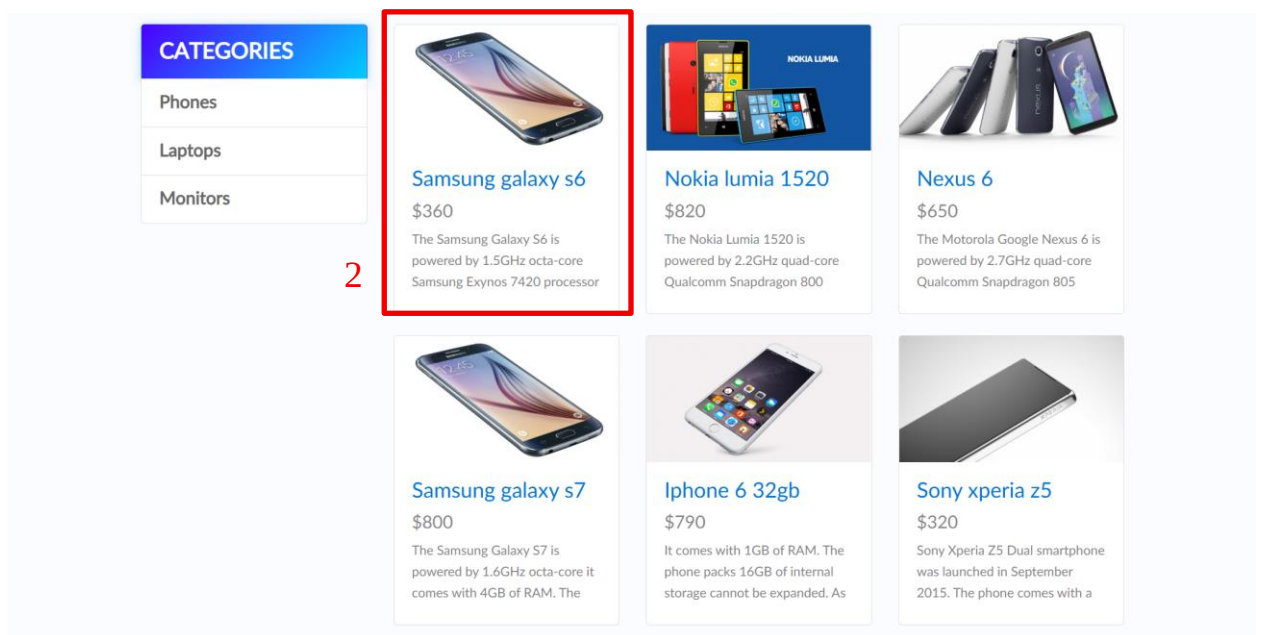


Рисунок 5 - Открытие страницы товара. Часть 2

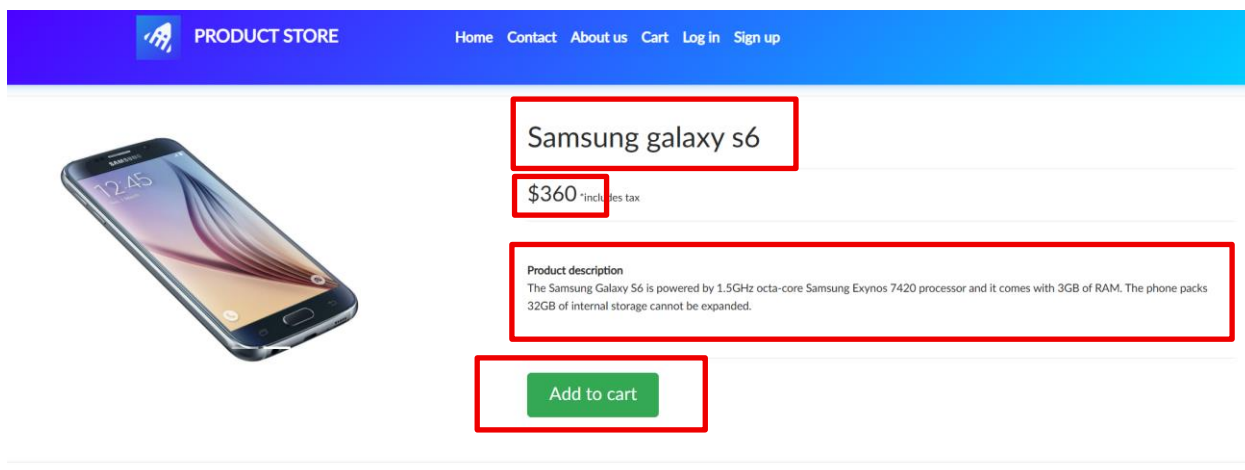


Рисунок 6 - Открытие страницы товара. Часть 3

На рисунке 6 выделены элементы карточки товара, наличие которых проверяется в результате выполнения теста.

Тест №3: Проверка перехода в корзину через верхнее меню

Цель: проверить корректность перехода в раздел корзины через верхнее меню сайта Demoblaze.

Ожидаемый результат: после нажатия на пункт меню Cart открывается страница корзины. На странице отображается таблица корзины с колонками Pic, Title, Price, x, а также кнопка Place Order.

В ходе выполнения теста пользователь открывает главную страницу сайта Demoblaze и нажимает пункт меню Cart. После перехода в раздел корзины проверяется, что страница открыта корректно: отображается таблица корзины, присутствуют основные колонки и доступна кнопка Place Order, необходимая для дальнейшего оформления заказа.

Результаты действий представлены на рисунках (см. рисунок 7, 8).

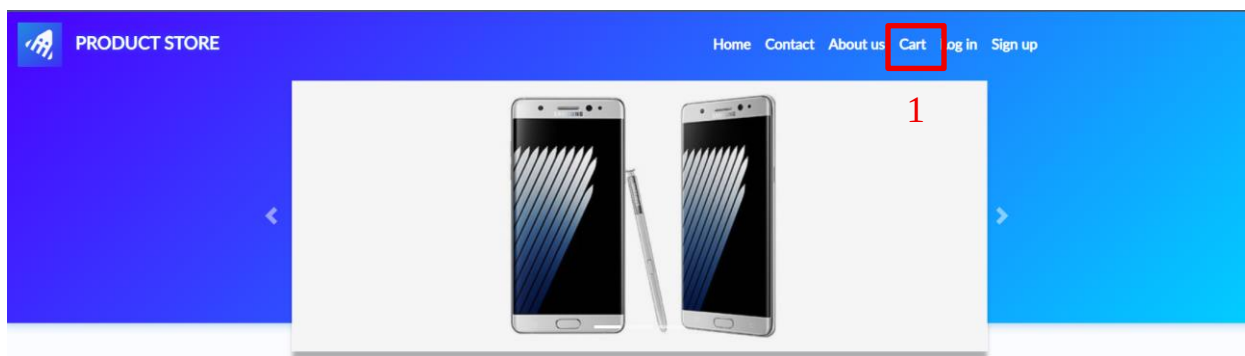


Рисунок 7 - Переход в корзину через верхнее меню. Часть 1

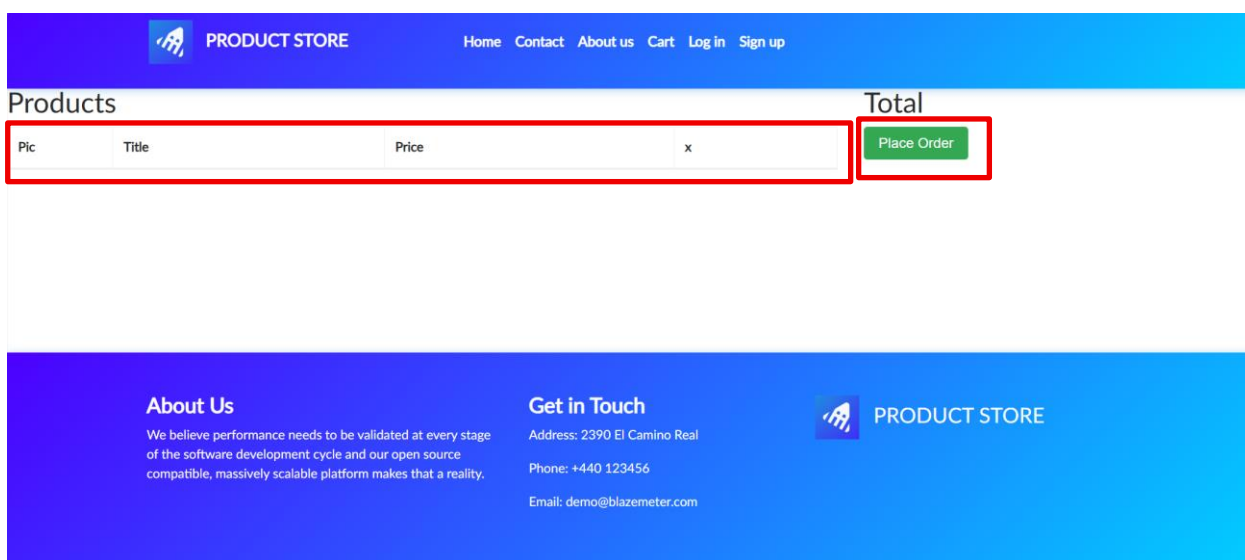


Рисунок 8 - Переход в корзину через верхнее меню. Часть 2

Тест №4: Проверка добавления товара в корзину

Цель: проверить корректность добавления товара в корзину на сайте Demoblaze.

Ожидаемый результат: после нажатия кнопки Add to cart появляется сообщение Product added. После перехода в раздел Cart добавленный товар Samsung Galaxy S6 отображается в таблице корзины.

В ходе выполнения теста пользователь открывает главную страницу сайта Demoblaze, переходит в категорию Phones и выбирает товар Samsung Galaxy S6. После открытия карточки товара пользователь нажимает кнопку Add to cart. В результате должно появиться всплывающее сообщение Product added, подтверждающее добавление товара. Затем пользователь закрывает

сообщение, переходит в раздел Cart и проверяет, что товар Samsung Galaxy S6 отображается в корзине.

Результаты действий представлены на рисунках (см. рисунки 9 – 12).

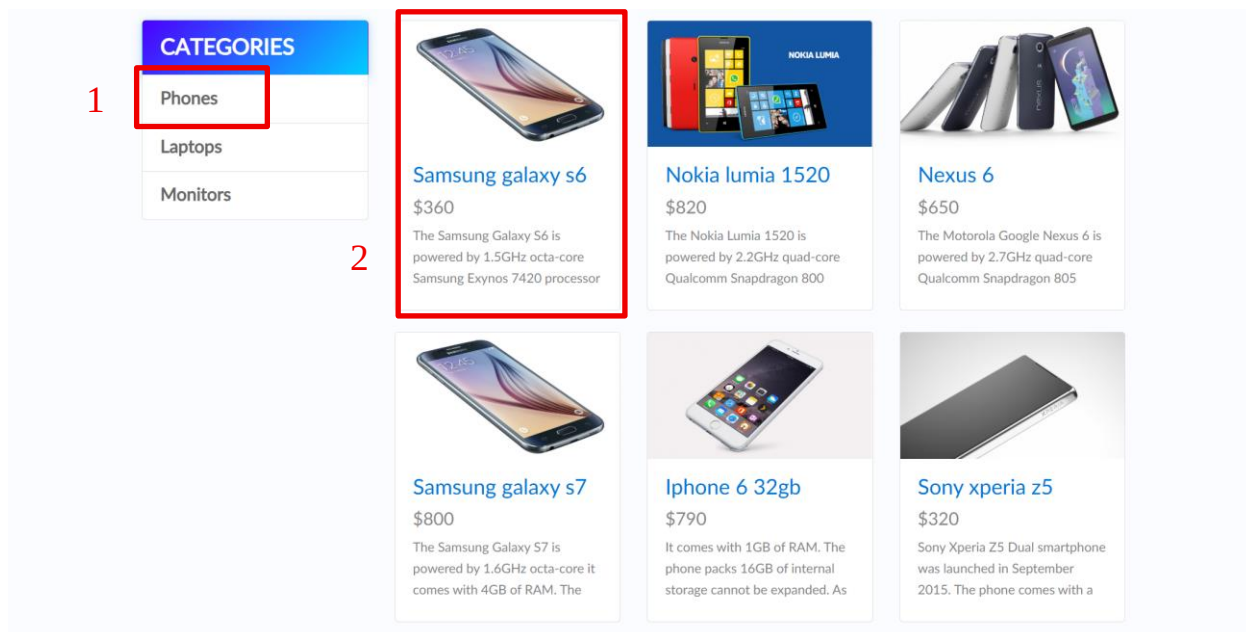


Рисунок 9 - Добавление товара в корзину. Часть 1

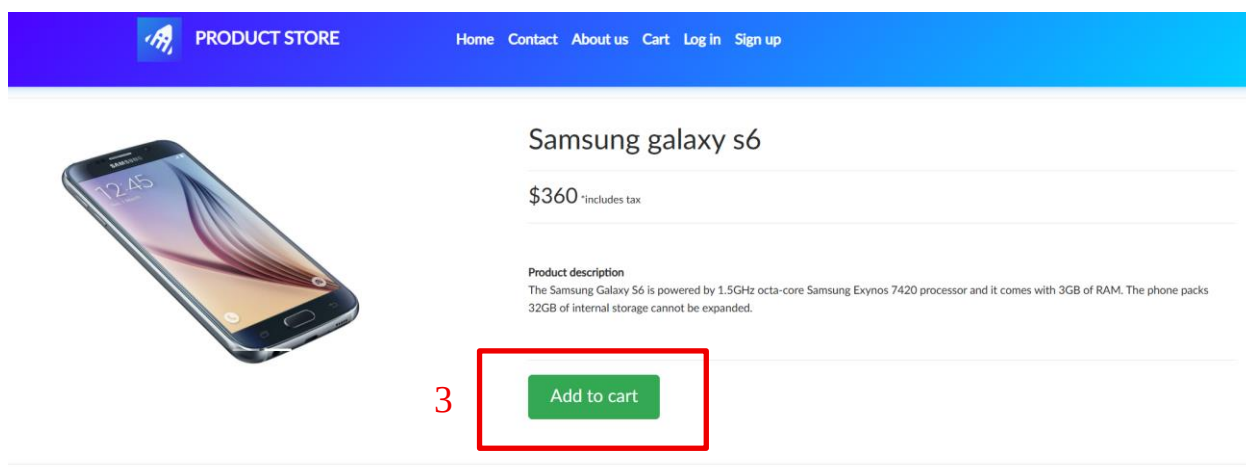


Рисунок 10 - Добавление товара в корзину. Часть 2

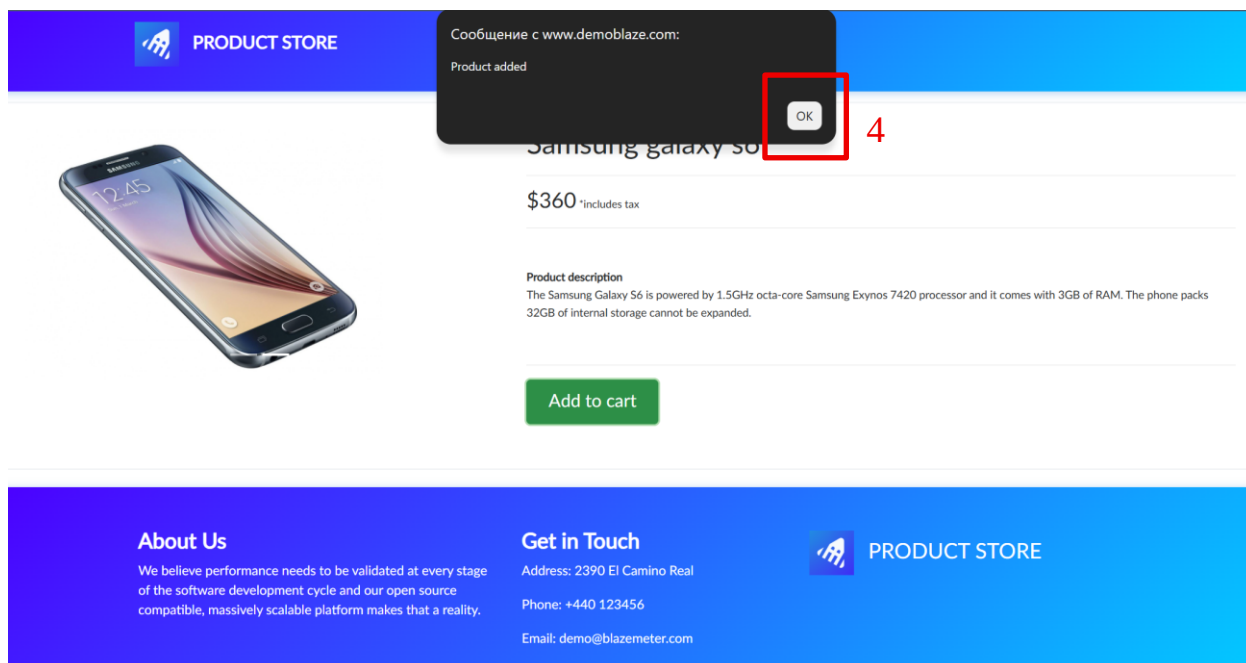


Рисунок 11 - Добавление товара в корзину. Часть 3

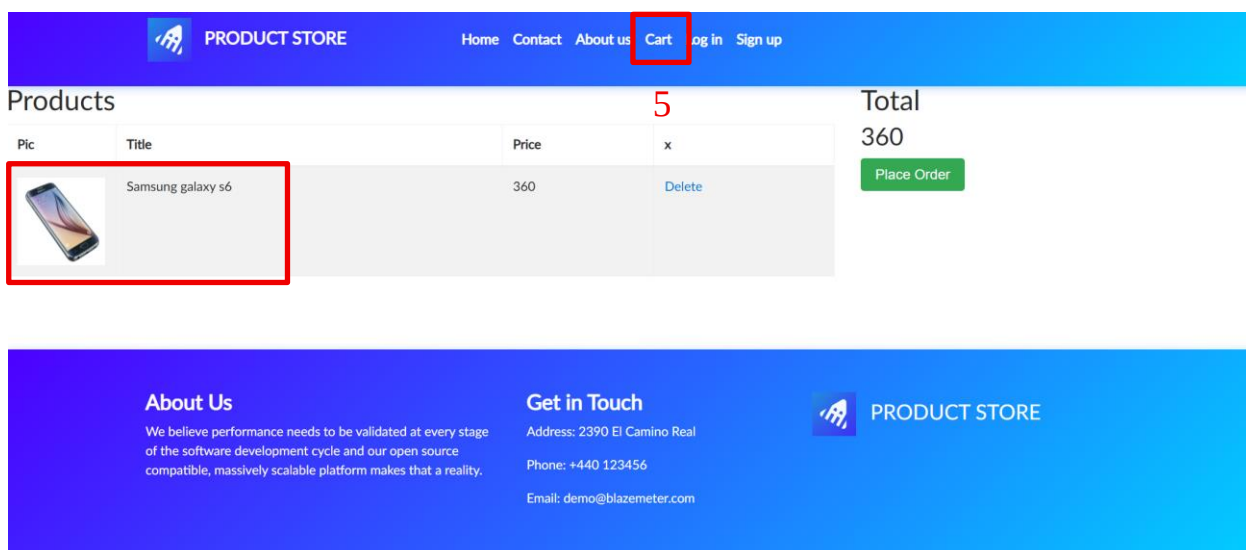


Рисунок 12 - Отображение добавленного товара в корзине

Тест №5: Проверка добавления нескольких товаров в корзину

Цель: проверить возможность добавления нескольких товаров в корзину сайта Demoblaze.

Ожидаемый результат: после последовательного добавления двух товаров в корзину отображаются оба выбранных товара: Samsung Galaxy S6 и Nokia Lumia 1520.

В ходе выполнения теста пользователь открывает сайт Demoblaze, переходит в категорию Phones и выбирает товар Samsung Galaxy S6. После открытия карточки товара пользователь нажимает кнопку Add to cart и закрывает сообщение Product added. Затем пользователь возвращается на главную страницу, снова переходит в категорию Phones, выбирает товар Nokia Lumia 1520 и также добавляет его в корзину. После перехода в раздел Cart проверяется, что в таблице корзины отображаются оба добавленных товара. Данный тест позволяет убедиться, что корзина корректно сохраняет несколько товаров, добавленных пользователем по очереди.

Результаты действий представлены на рисунках (см. рисунки 13 – 16).

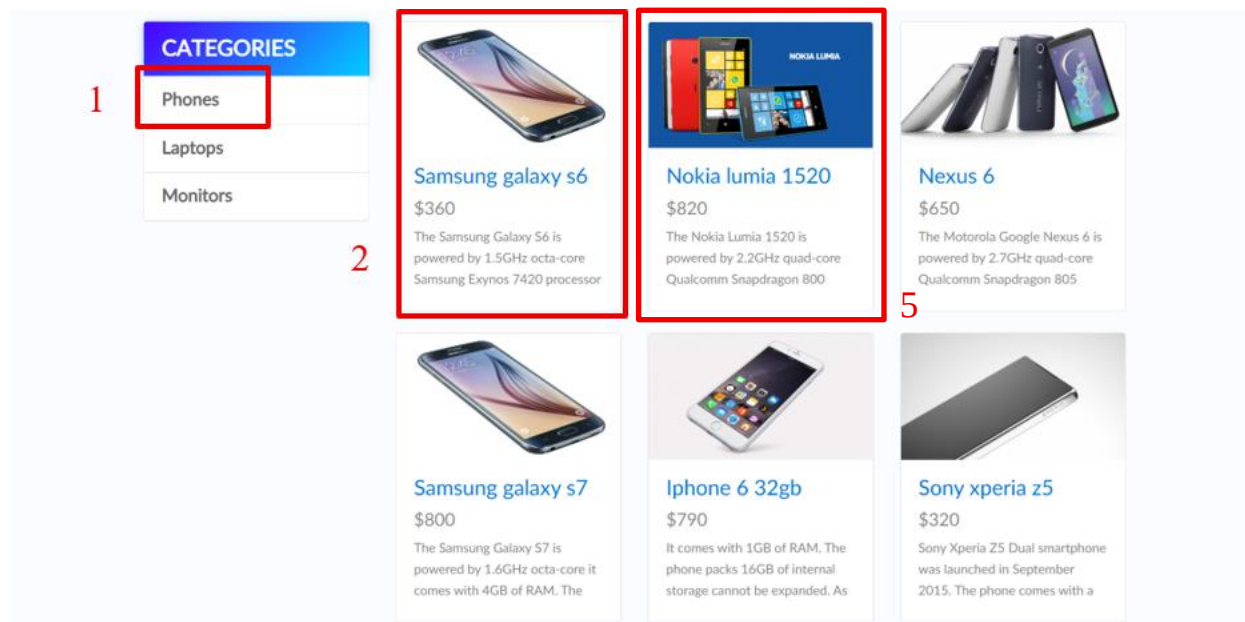


Рисунок 13 - Добавление нескольких товаров в корзину. Часть 1

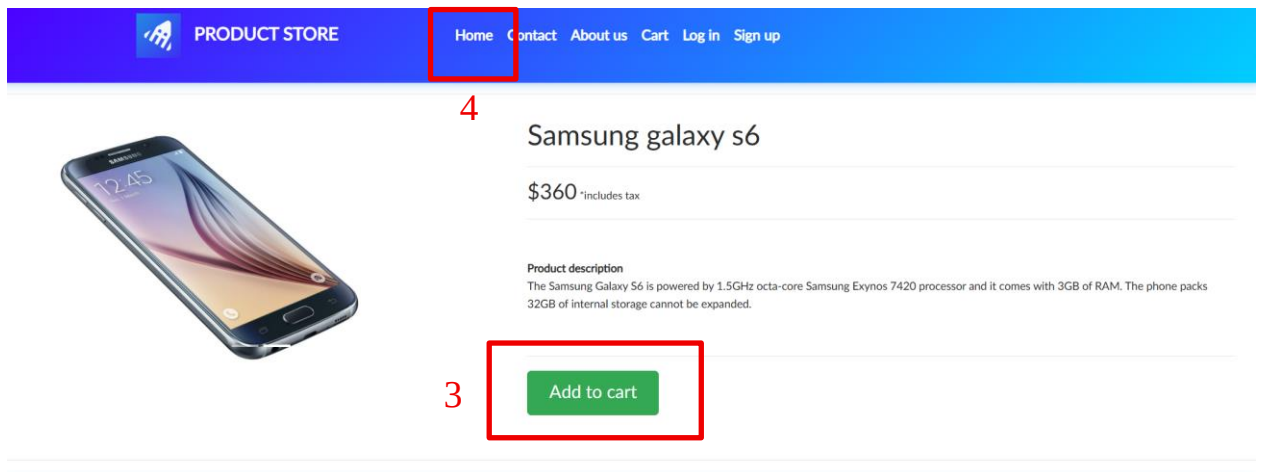


Рисунок 14 - Добавление нескольких товаров в корзину. Часть 2

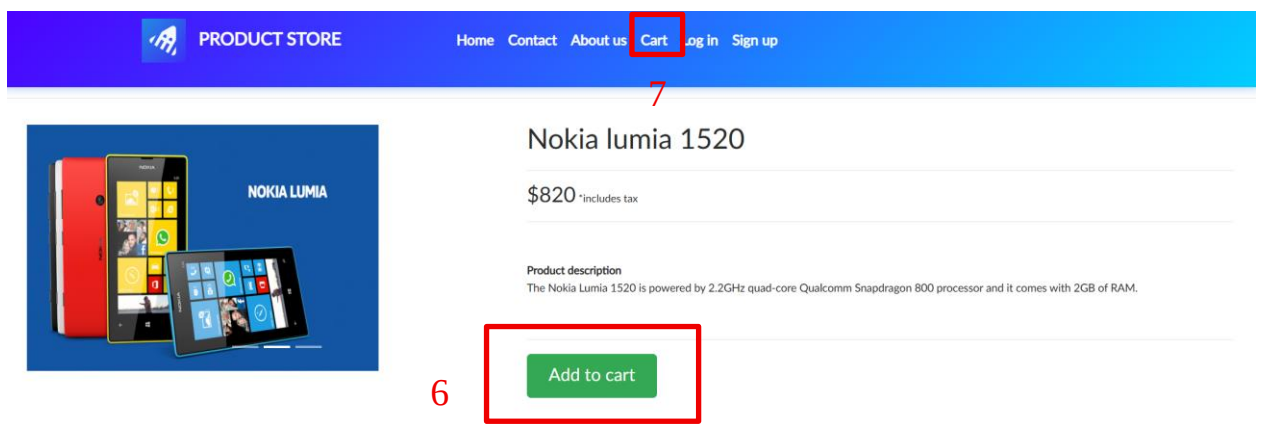


Рисунок 15 - Добавление нескольких товаров в корзину. Часть 3

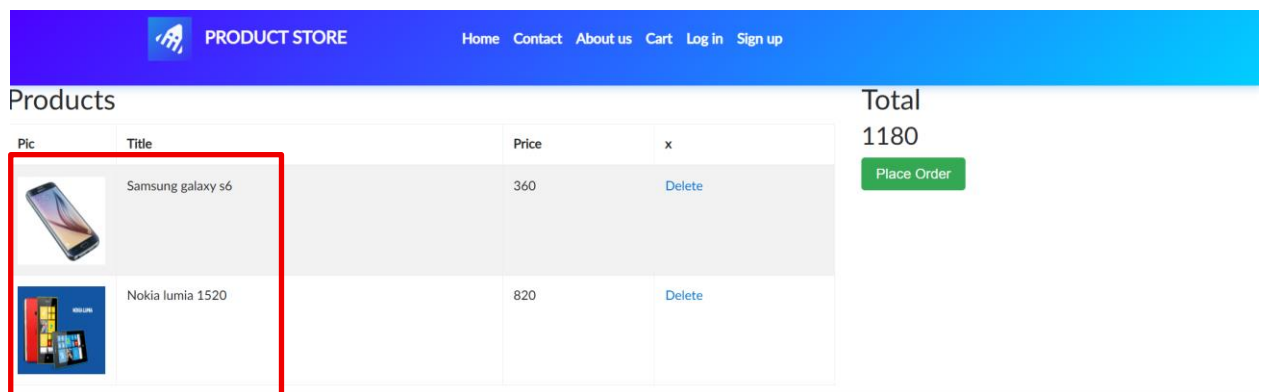


Рисунок 16 - Отображение нескольких товаров в корзине

Тест №6: Проверка удаления товара из корзины

Цель: проверить возможность удаления товара из корзины на сайте Demoblaze.

Ожидаемый результат: после нажатия ссылки Delete товар Samsung Galaxy S6 удаляется из корзины и больше не отображается в таблице товаров.

В ходе выполнения теста пользователь открывает сайт Demoblaze, переходит в категорию Phones и выбирает товар Samsung Galaxy S6. После открытия карточки товара пользователь нажимает кнопку Add to cart и закрывает сообщение Product added. Затем выполняется переход в раздел Cart, где отображается добавленный товар. После нажатия ссылки Delete напротив товара Samsung Galaxy S6 товар удаляется из корзины. Данный тест позволяет проверить, что функция удаления товара из корзины работает корректно.

Результаты действий представлены на рисунках (см. рисунки 17 – 21).

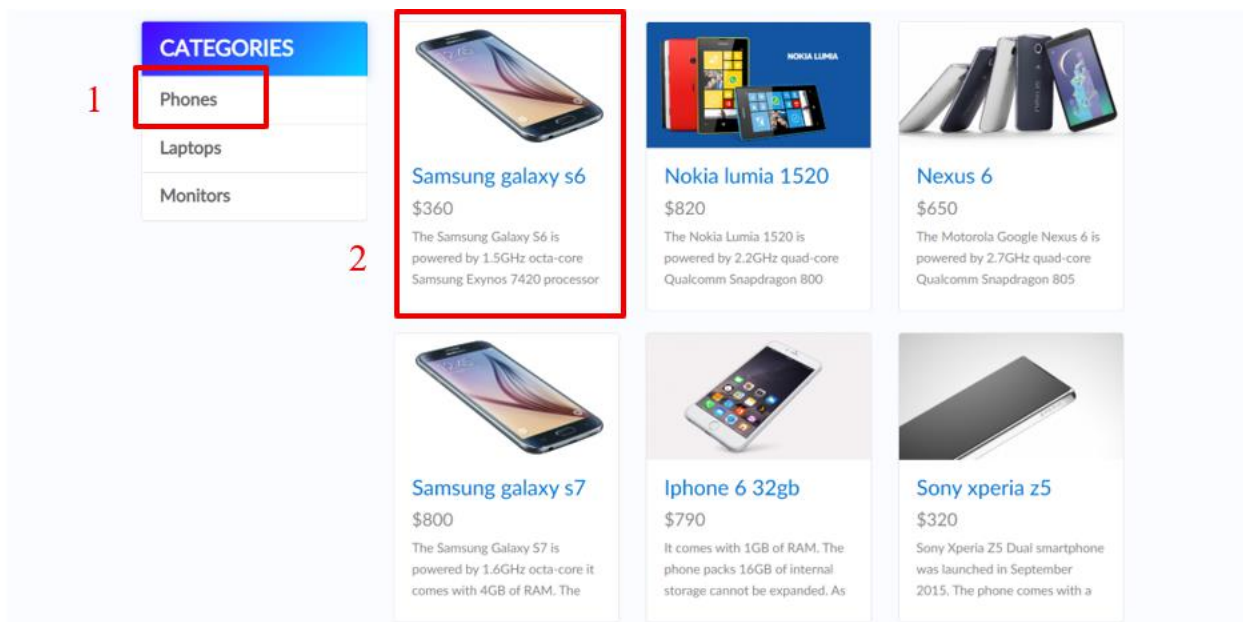


Рисунок 17 - Удаление товара из корзины. Часть 1

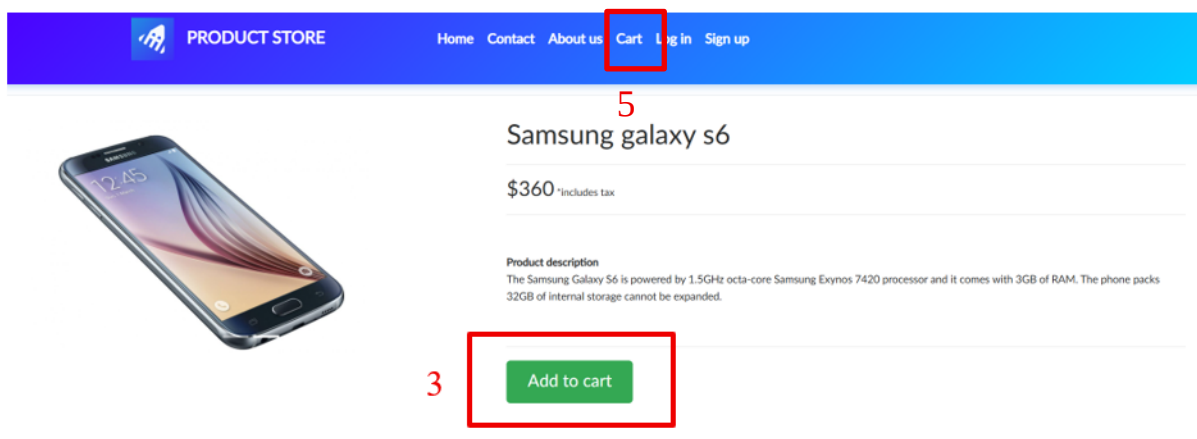


Рисунок 18 - Удаление товара из корзины. Часть 2

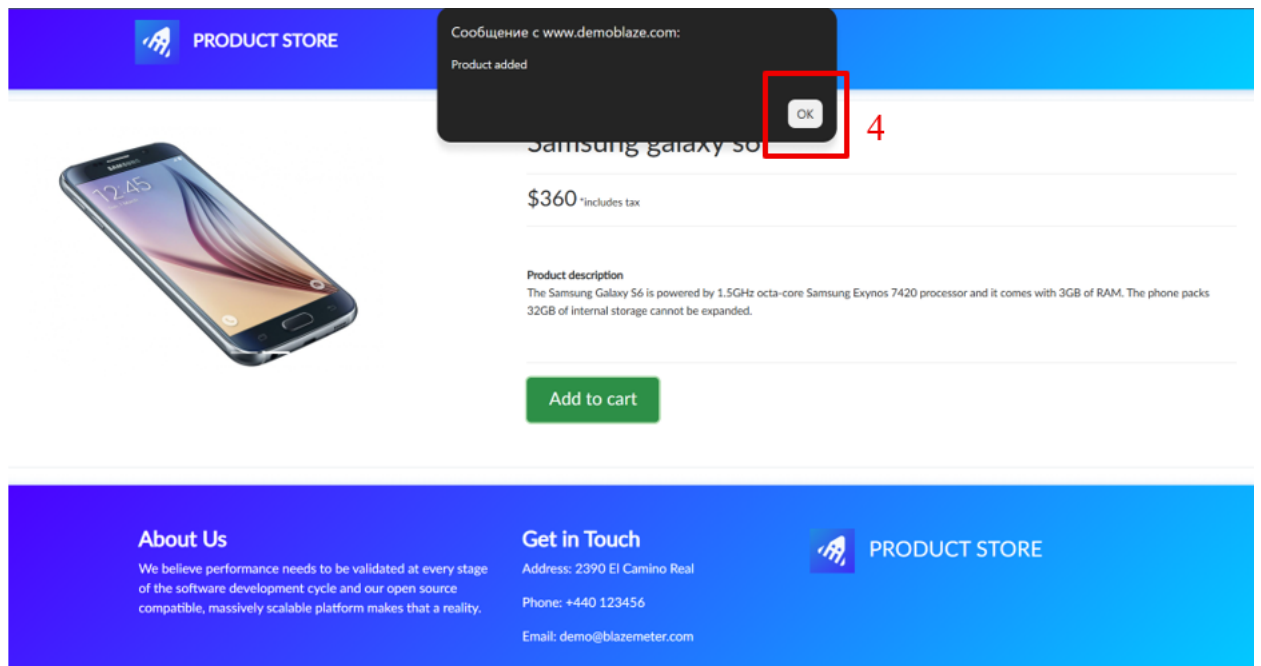


Рисунок 19 - Удаление товара из корзины. Часть 3

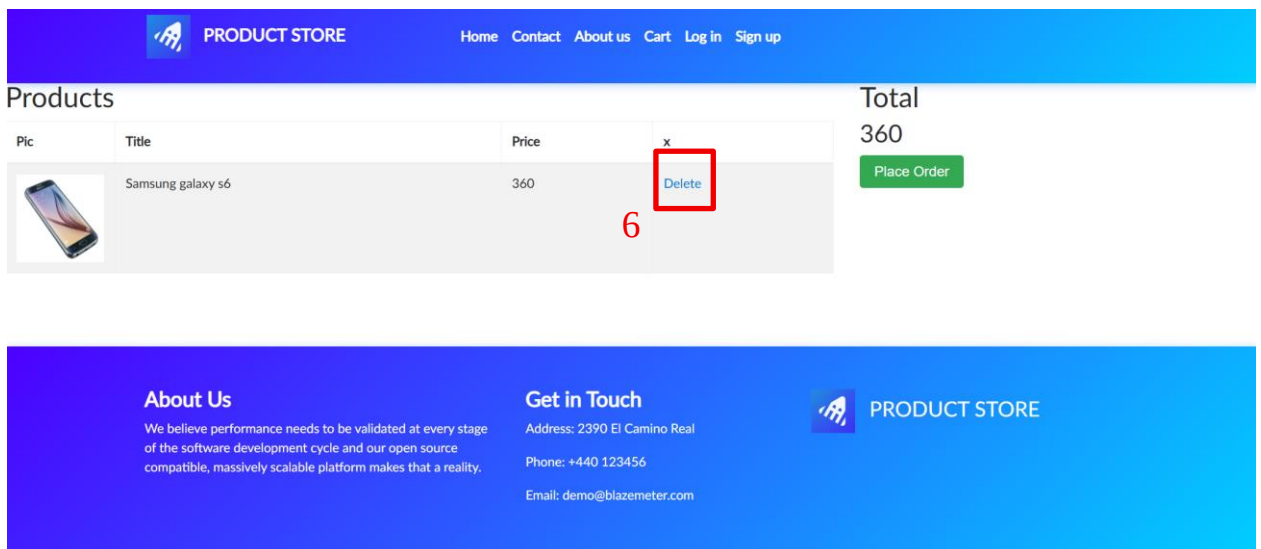


Рисунок 20 - Удаление товара из корзины. Часть 4

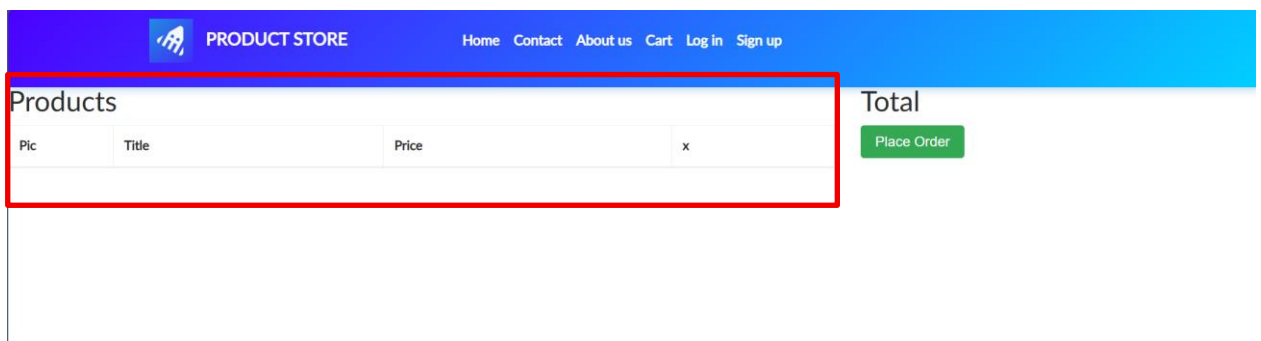


Рисунок 21 - Удаление товара из корзины. Часть 5

Тест №7: Проверка оформления заказа

Цель: проверить открытие формы оформления заказа и возможность оформления покупки с корректными данными.

Ожидаемый результат: после нажатия кнопки Place Order открывается форма оформления заказа Place order. После заполнения полей корректными данными и нажатия кнопки Purchase пользователю отображается сообщение Thank you for your purchase!.

В ходе выполнения теста пользователь открывает сайт Demoblaze, переходит в категорию Phones и выбирает товар Samsung Galaxy S6. После открытия карточки товара пользователь добавляет товар в корзину, закрывает сообщение Product added и переходит в раздел Cart. В корзине пользователь нажимает кнопку Place Order, после чего открывается форма оформления заказа. Затем поля формы заполняются корректными данными: Test User, Russia, Saint Petersburg, 1234567890, 06 и 2026. После нажатия кнопки Purchase проверяется появление сообщения об успешном оформлении заказа. Данный тест позволяет убедиться, что пользователь может перейти к оформлению заказа, заполнить форму и завершить покупку.

Результаты действий представлены на рисунках (см. рисунки 22 – 27).

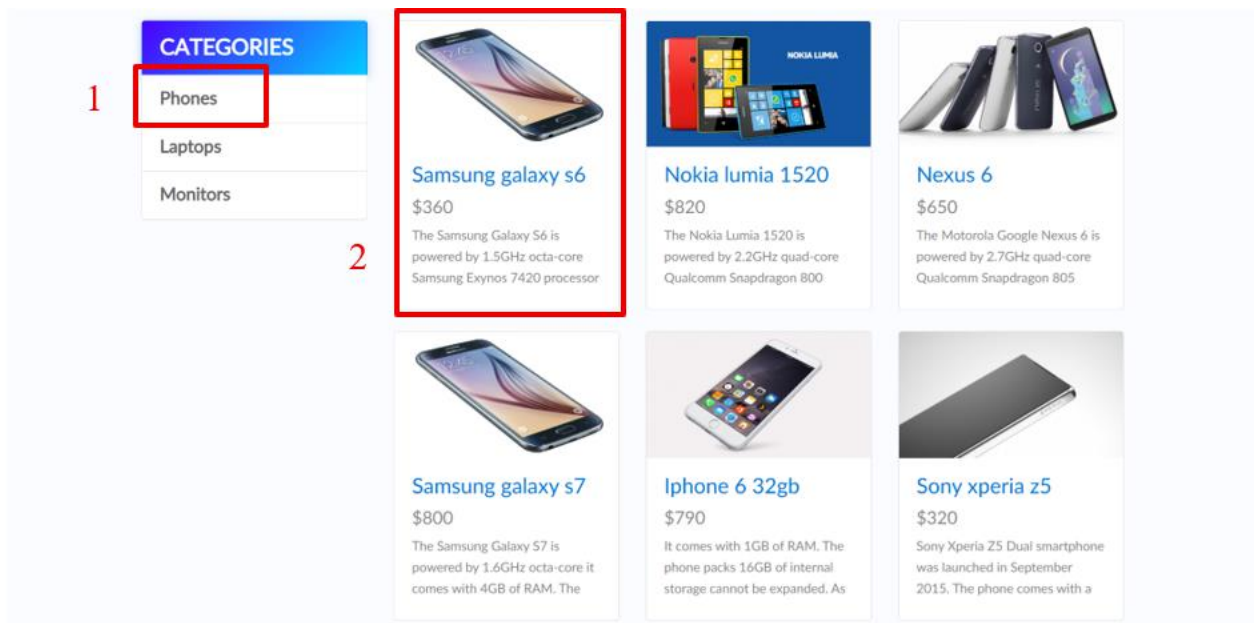


Рисунок 22 - Оформление заказа. Часть 1

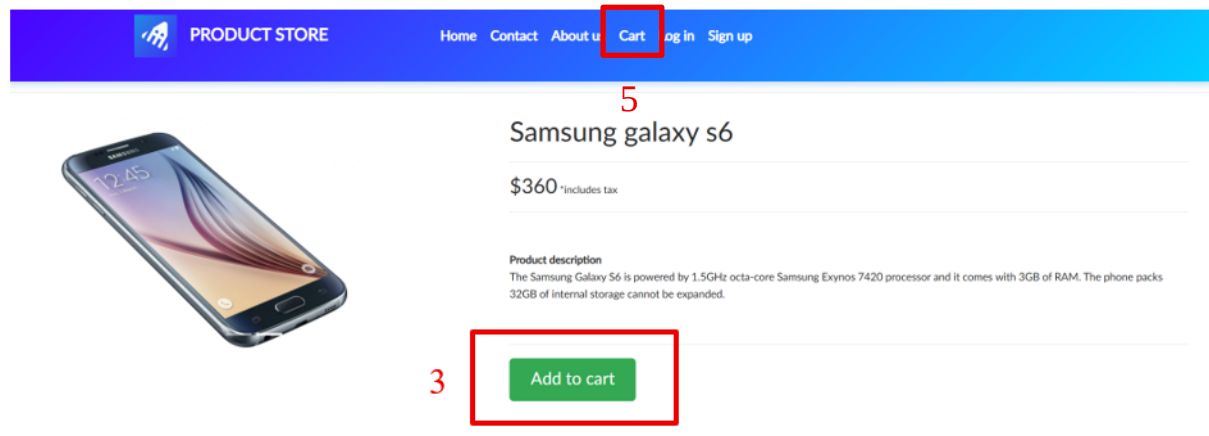


Рисунок 23 - Оформление заказа. Часть 2

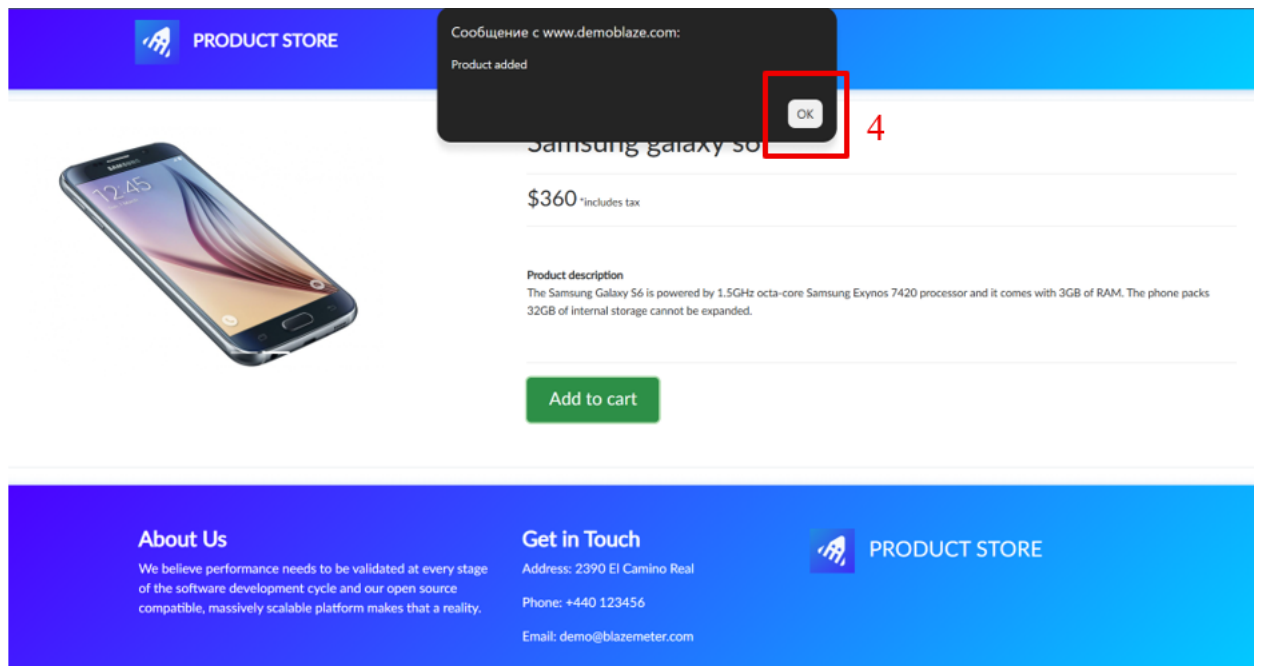


Рисунок 24 - Оформление заказа. Часть 3

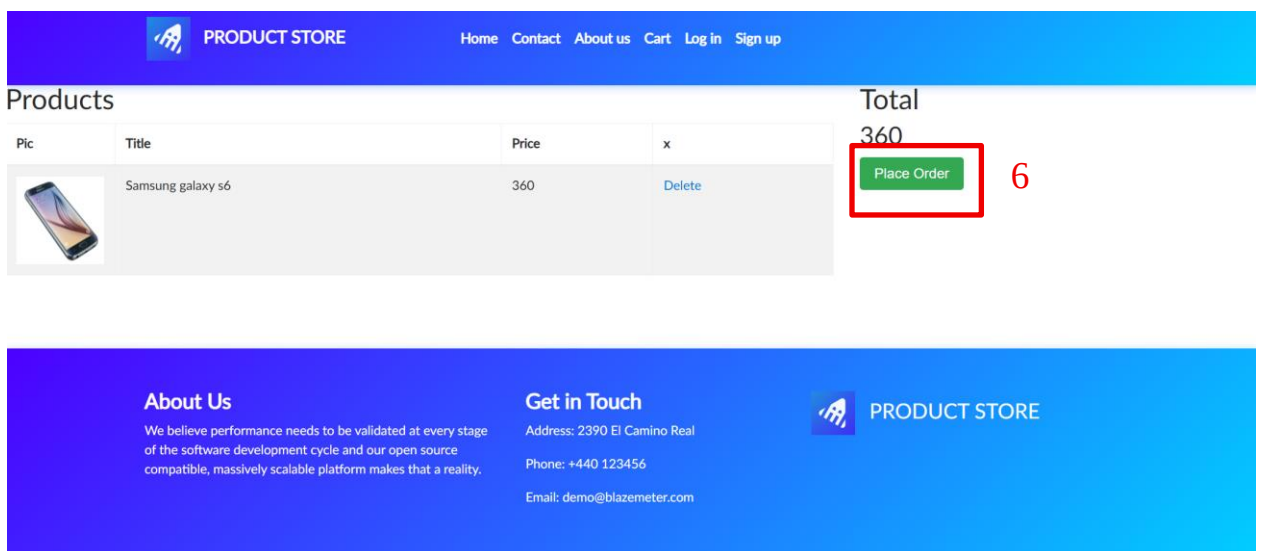


Рисунок 25 - Оформление заказа. Часть 4

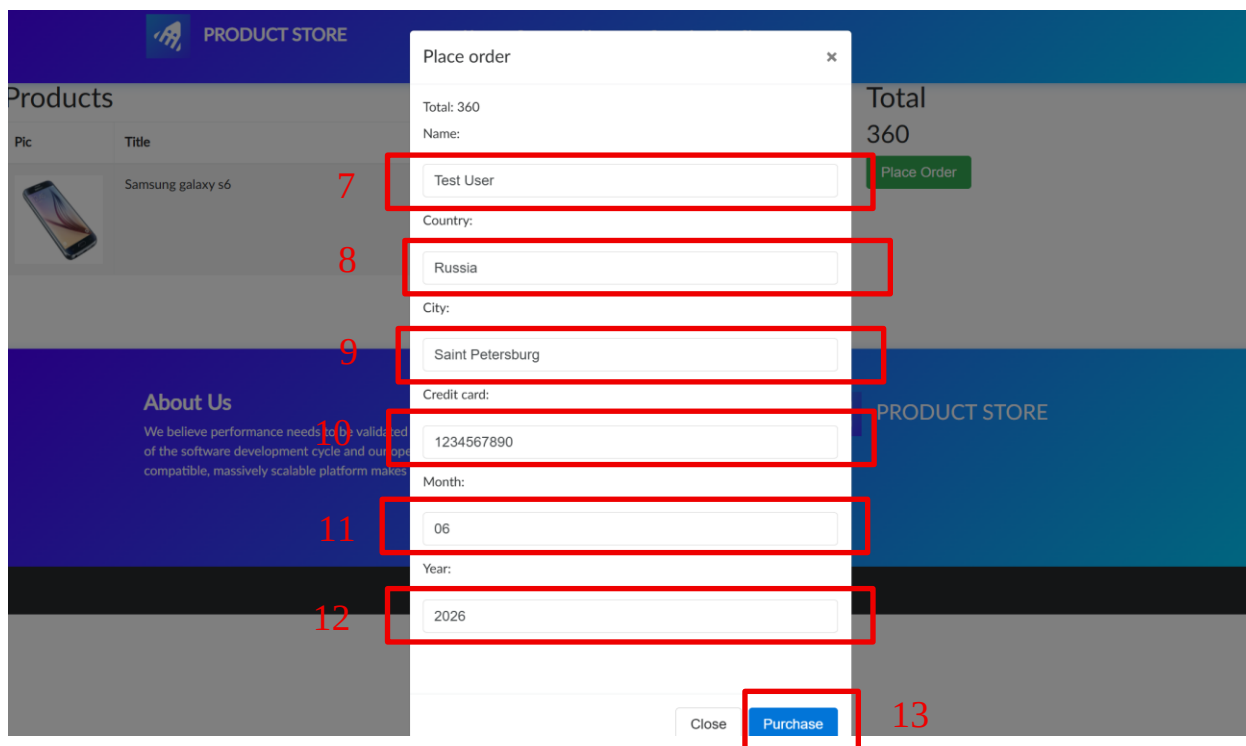


Рисунок 26 - Оформление заказа. Часть 5

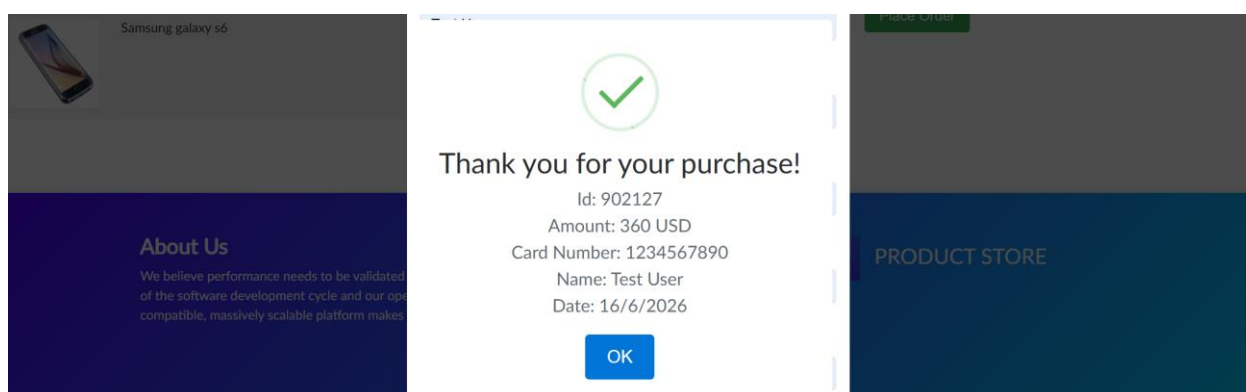


Рисунок 27 - Сообщение об успешном оформлении заказа

Тест №8: Проверка отмены оформления заказа

Цель: проверить возможность закрытия формы оформления заказа без завершения покупки.

Ожидаемый результат: после нажатия кнопки Close форма оформления заказа закрывается, покупка не оформляется, а товар Samsung Galaxy S6 остается в корзине.

В ходе выполнения теста пользователь открывает сайт Demoblaze, переходит в категорию Phones и выбирает товар Samsung Galaxy S6. После

открытия карточки товара пользователь добавляет товар в корзину, закрывает сообщение Product added и переходит в раздел Cart. В корзине пользователь нажимает кнопку Place Order, после чего открывается форма оформления заказа. Затем пользователь закрывает форму с помощью кнопки Close. После закрытия формы проверяется, что оформление покупки не было завершено, а товар Samsung Galaxy S6 по-прежнему отображается в корзине.

Результаты действий представлены на рисунках (см. рисунки 28 – 33).

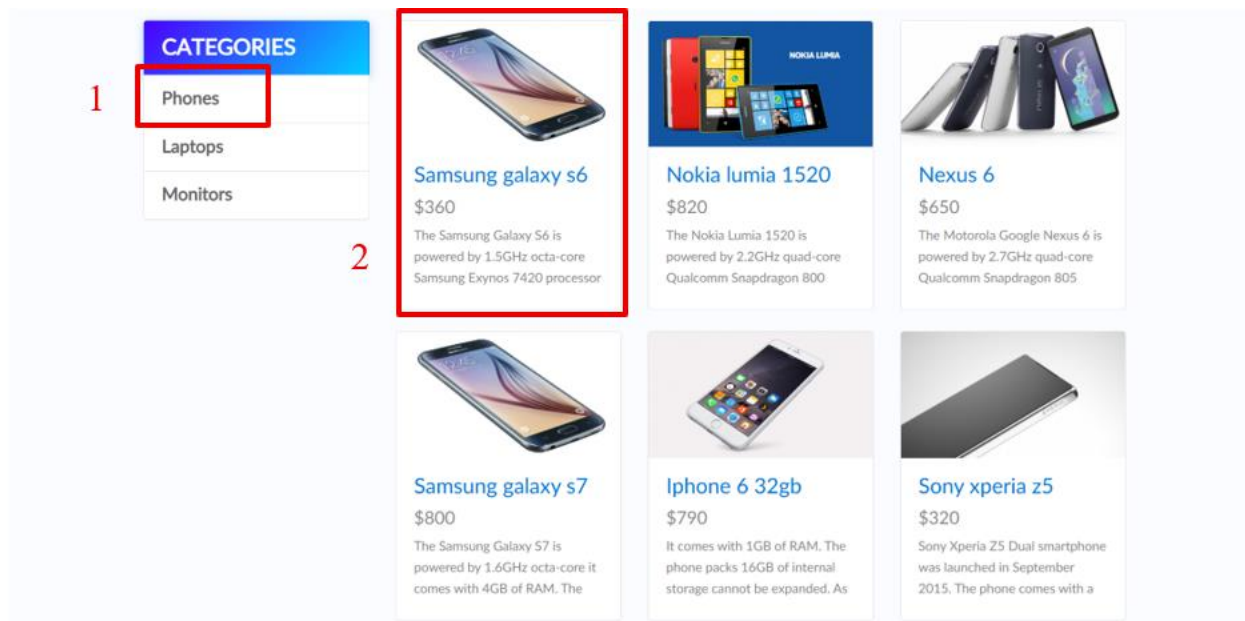


Рисунок 28 - Отмена оформления заказа. Часть 1

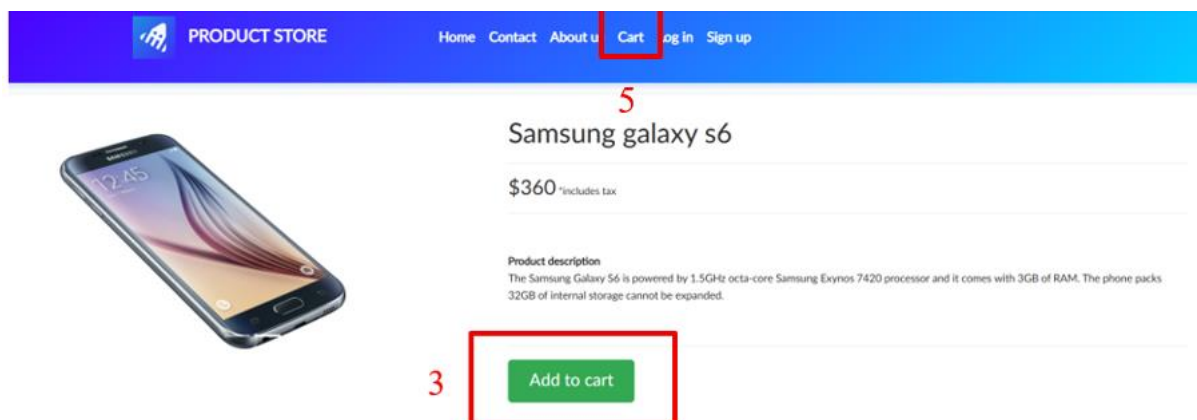


Рисунок 29 - Отмена оформления заказа. Часть 2

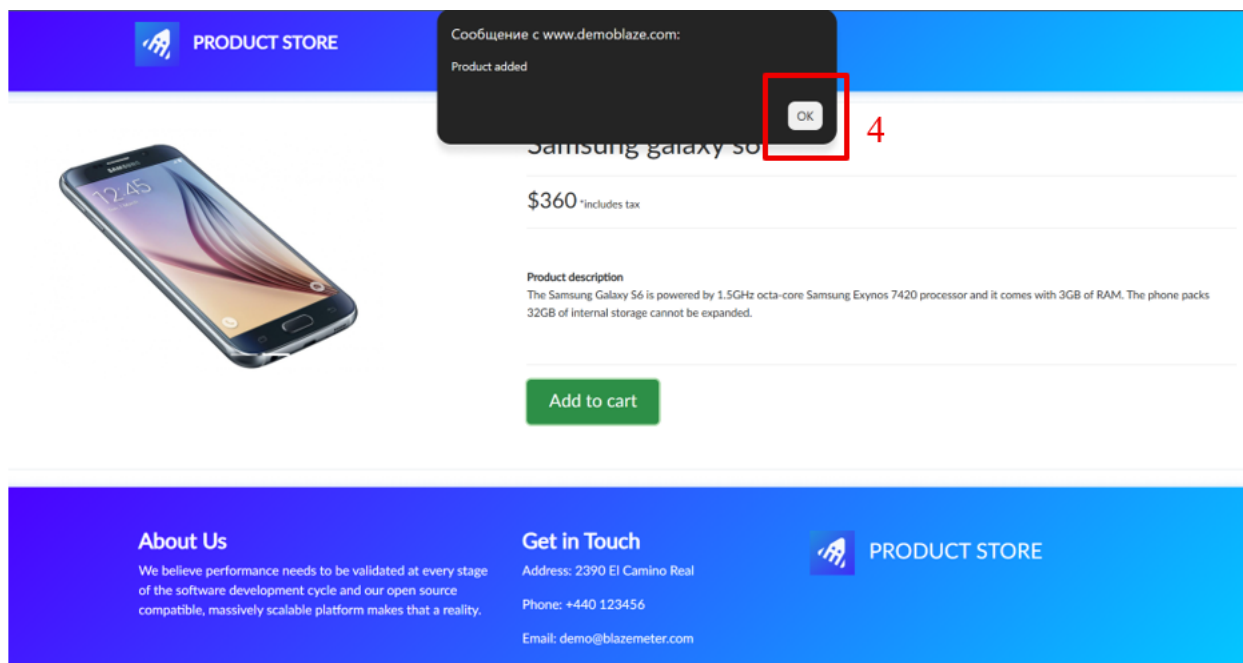


Рисунок 30 - Отмена оформления заказа. Часть 3

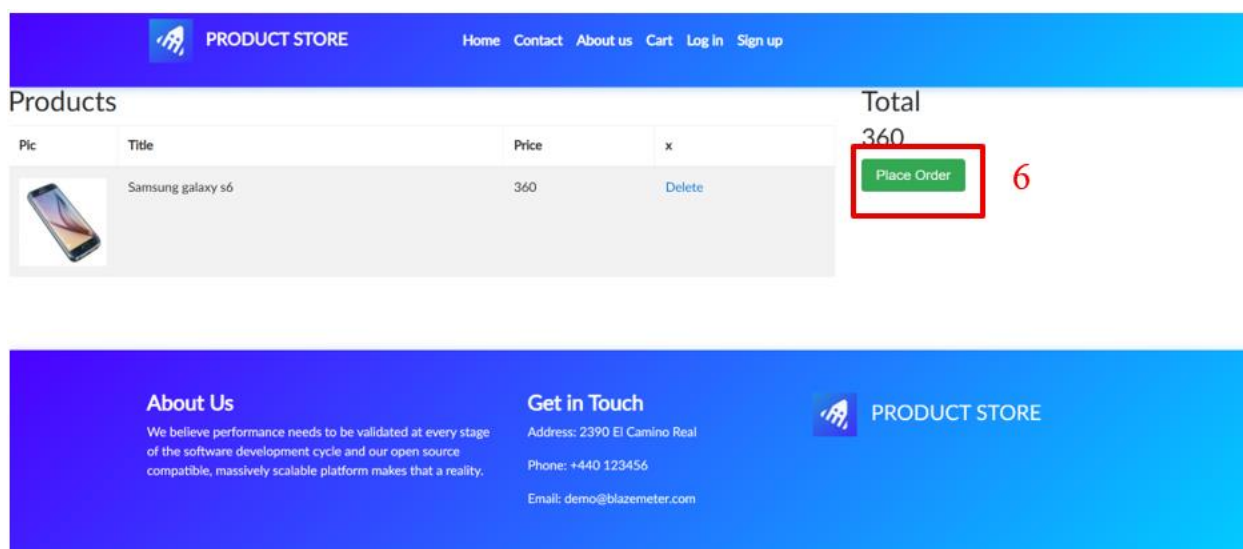


Рисунок 31 - Отмена оформления заказа. Часть 4

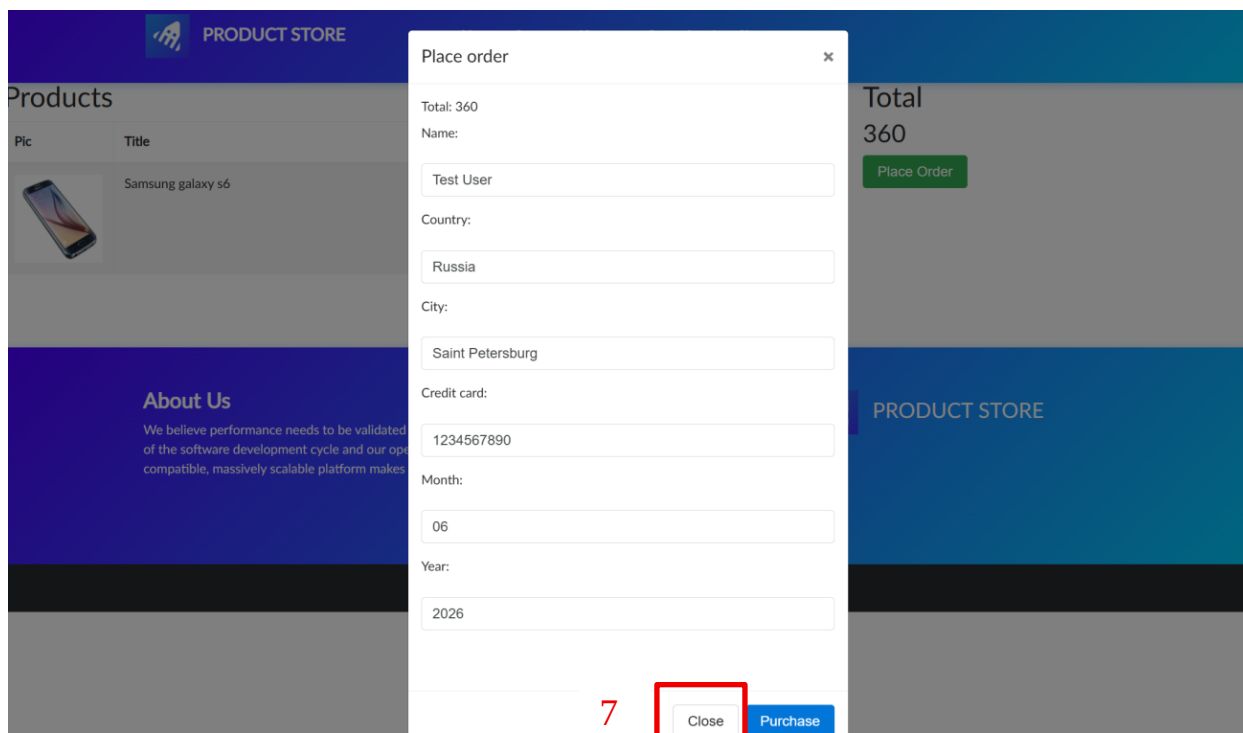


Рисунок 32 - Отмена оформления заказа. Часть 5

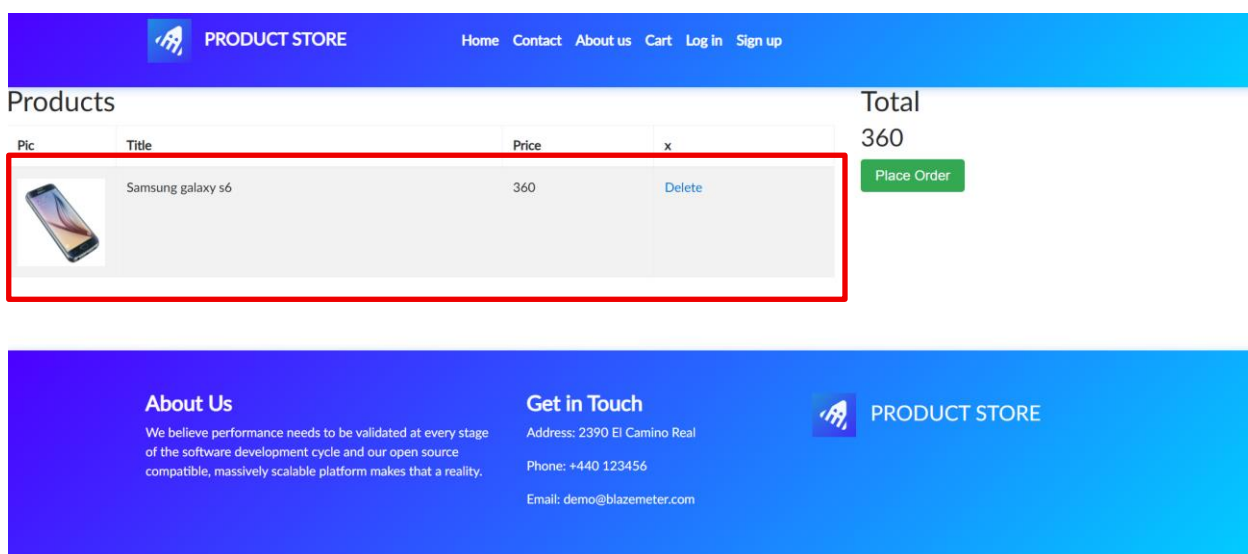


Рисунок 33 - Корзина после отмены оформления заказа

Товар остался в корзине после закрытия формы

Тест №9: Проверка отображения пустой корзины

Цель: проверить отображение корзины без добавленных товаров, а также появление товара в корзине после его добавления.

Ожидаемый результат: при переходе в раздел Cart без добавления товаров таблица корзины остается пустой. После добавления товара Samsung Galaxy S6 и повторного перехода в Cart товар отображается в таблице корзины.

В ходе выполнения теста пользователь сначала открывает сайт Demoblaze и переходит в раздел Cart, чтобы проверить состояние пустой корзины. После этого выполняется добавление товара Samsung Galaxy S6: пользователь переходит в категорию Phones, открывает карточку товара, нажимает кнопку Add to cart, закрывает сообщение Product added и снова переходит в раздел Cart. В результате проверяется, что после добавления товара пустая корзина изменила состояние, и в ней отображается Samsung Galaxy S6.

Результаты действий представлены на рисунках (см. рисунки 34 – 39).

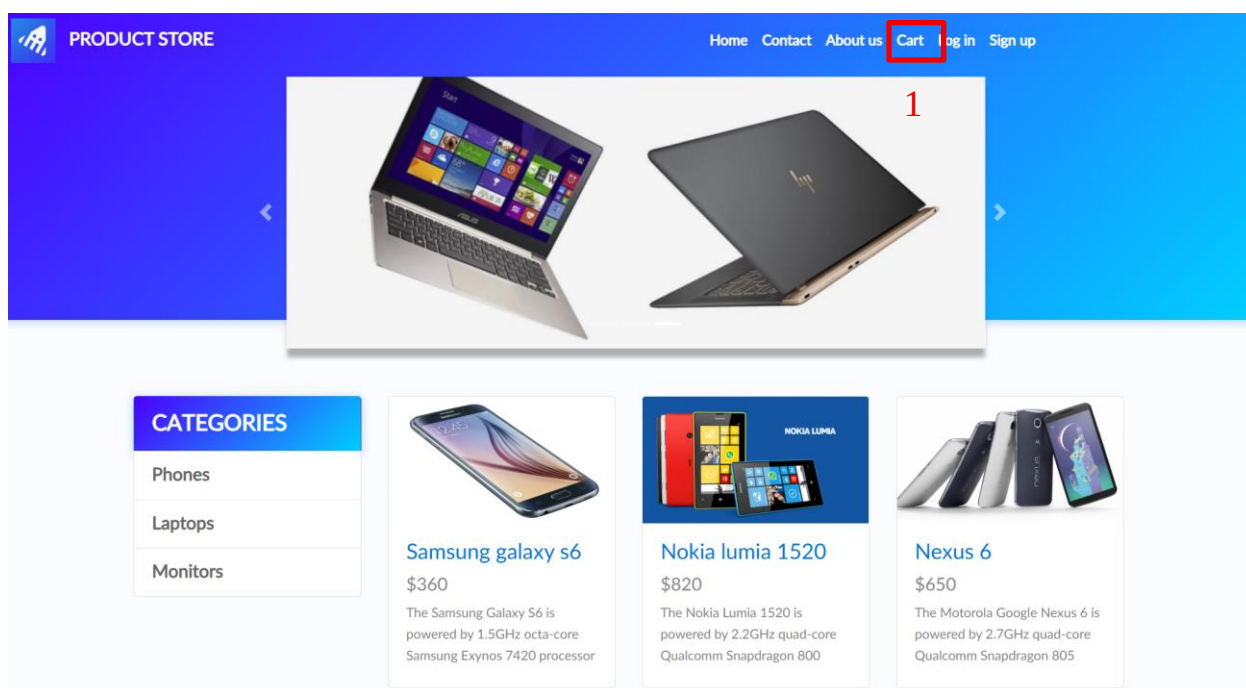


Рисунок 34 - Проверка отображения пустой корзины. Часть 1

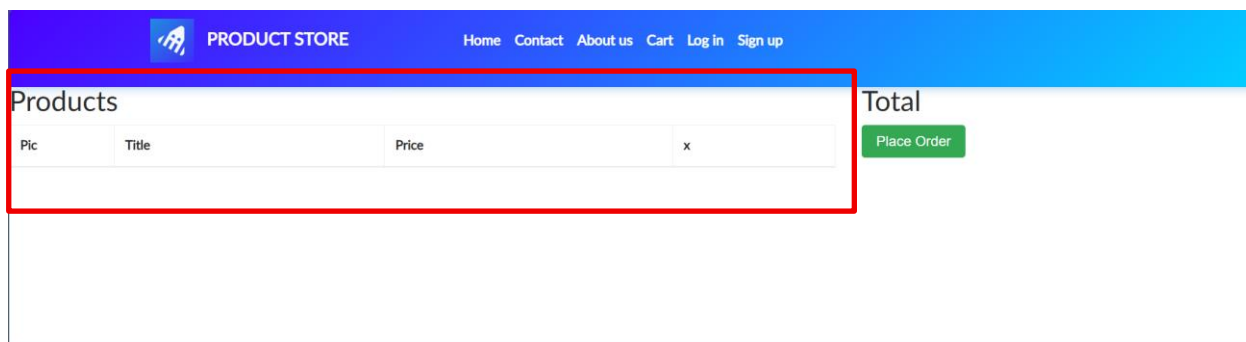


Рисунок 35 - Проверка отображения пустой корзины. Часть 2

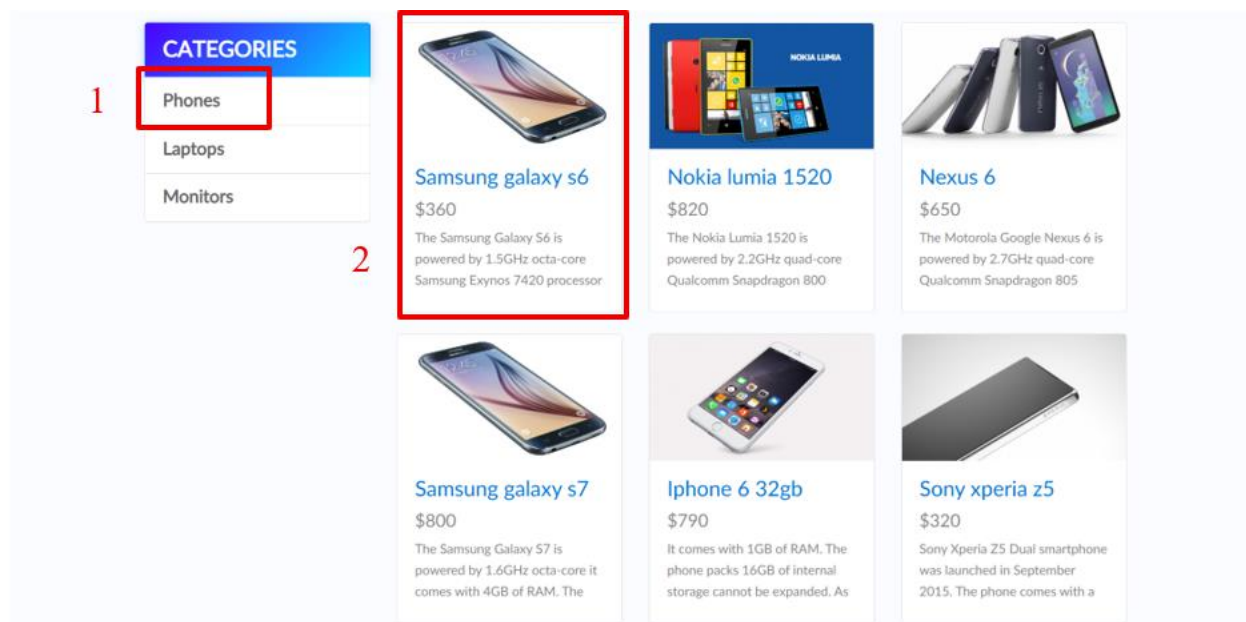


Рисунок 36 - Проверка отображения пустой корзины. Часть 3

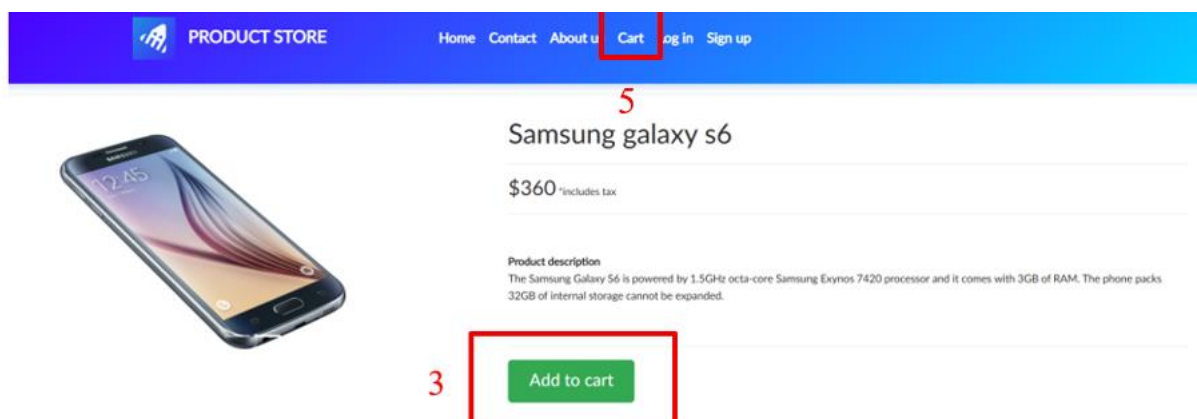


Рисунок 37 - Проверка отображения пустой корзины. Часть 4

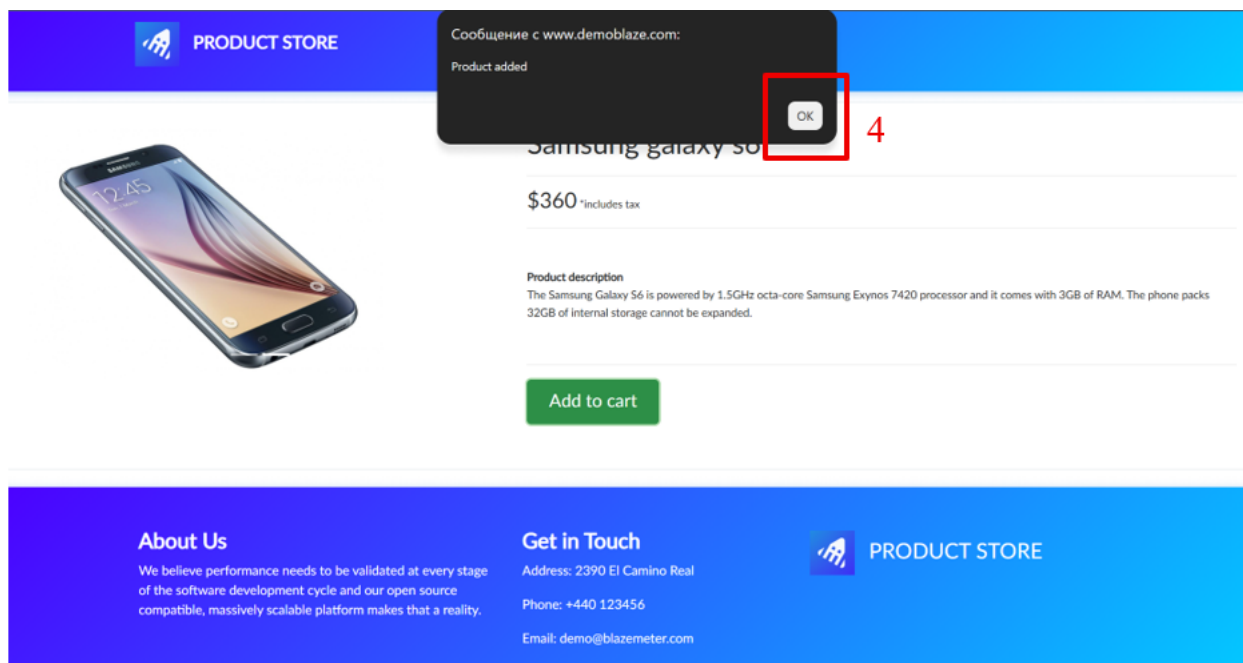


Рисунок 38 - Проверка отображения пустой корзины. Часть 5

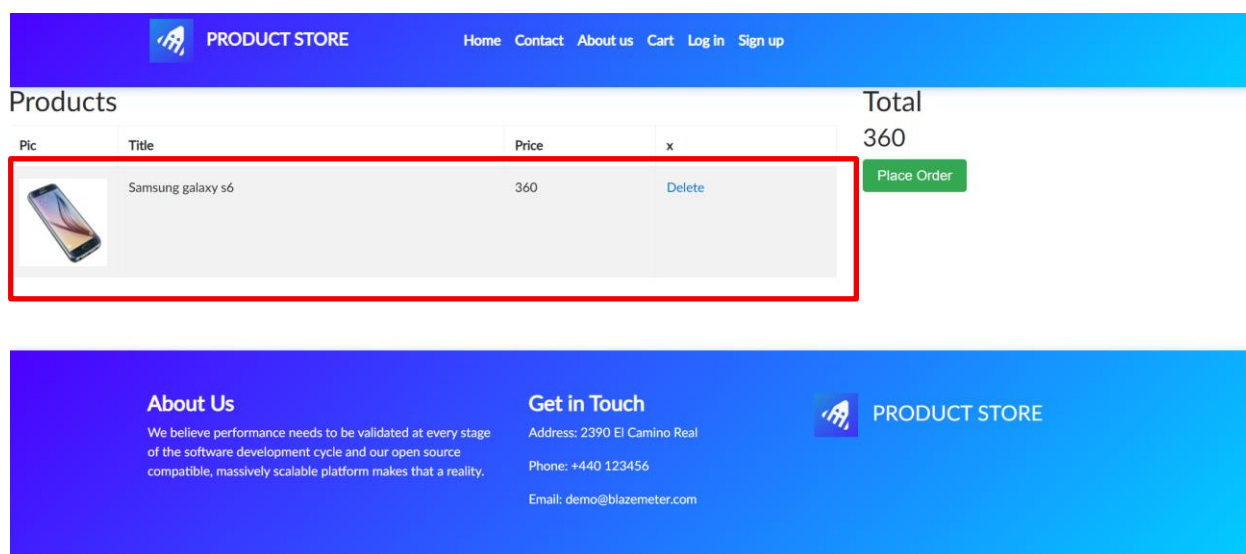


Рисунок 39 - Проверка отображения пустой корзины. Часть 6

Результаты выполнения тестов представлены на рис. 40

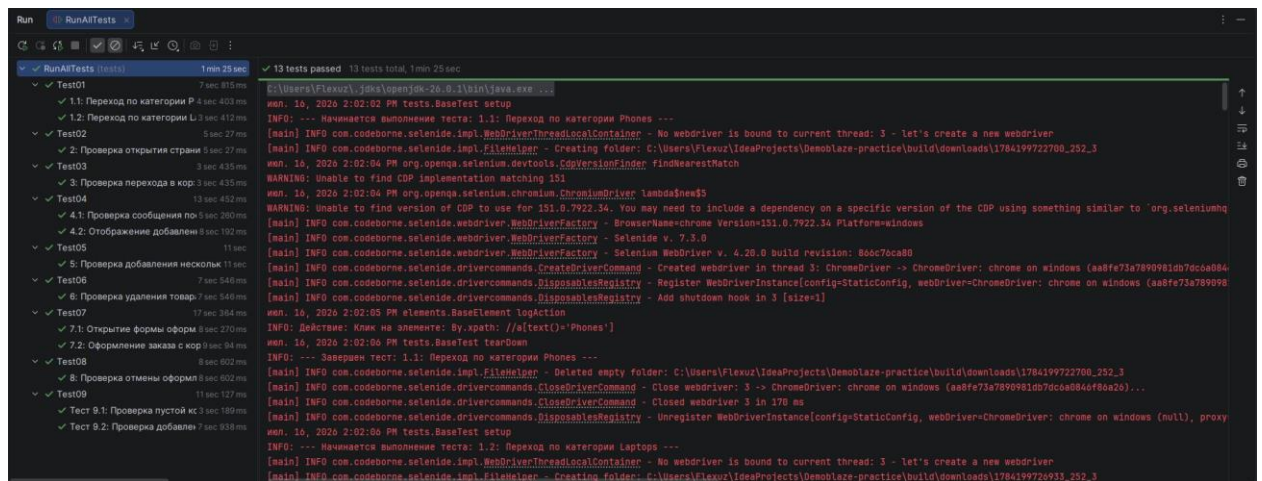


Рисунок 40 - результаты выполнения тестов

2 РЕЗУЛЬТАТЫ РАБОТЫ

В данном разделе приведено описание архитектуры разработанной системы автоматизированного тестирования для сайта Demoblaze. В основу проекта заложен паттерн проектирования Page Object Model (POM), который позволяет разделить логику тестов и описание элементов пользовательского интерфейса, обеспечивая высокую расширяемость и удобство поддержки кода. Проект реализован на языке Java с использованием библиотеки Selenide для взаимодействия с браузером и тестового фреймворка JUnit 5.

Программный комплекс разделен на четыре логических пакета в соответствии с архитектурой POM: элементы страниц (elements), страницы (pages), тесты (tests) и вспомогательные утилиты (utils). Классы тестов не имеют прямого доступа к элементам страниц и взаимодействуют с ними исключительно через методы классов страниц, что соответствует принципам паттерна Page Object Model.

2.1. Описание программных классов и их методов

Программный комплекс разделен на несколько логических пакетов: компоненты, страницы, тесты и вспомогательные утилиты.

Пакет elements (элементы страниц):

BaseElement — базовый класс для всех UI-элементов. Содержит защищенное поле baseElement типа SelenideElement и метод isDisplayed() для проверки видимости элемента на странице. Выступает основой для всех дочерних классов-элементов.

Button — класс для работы с кнопками. Наследуется от BaseElement и расширяет его функциональность методом click(), который выполняет нажатие на кнопку с предварительным логированием действия. Используется для всех интерактивных кнопок на сайте: категории товаров, "Add to cart", "Place Order", "Delete".

Input — класс для работы с полями ввода. Наследуется от BaseElement и предоставляет метод fill(String value) для заполнения поля указанным

значением. Применяется при работе с формами оформления заказа (поля Name, Country, City, Credit Card, Month, Year).

Пакет pages (страницы):

BasePage — базовый класс для всех страниц. Содержит метод waitForElement(String selector), который ожидает появления элемента на странице перед выполнением действий. Обеспечивает стабильность тестов при асинхронной загрузке контента.

MainPage — класс, представляющий главную страницу сайта Demoblaze. Отвечает за навигацию по каталогу товаров. Содержит методы: selectCategory() — выбор категории товаров (Phones, Laptops), openProduct() — открытие карточки товара по названию, goToCart() — переход в корзину, isProductVisible() — проверка наличия товара в списке, clickHome() — возврат на главную страницу, isTitleCorrect() — проверка заголовка страницы.

CartPage — класс, представляющий страницу корзины. Содержит методы для управления товарами: deleteProduct() — удаление товара из корзины, isProductPresent() — проверка наличия товара, isCartEmpty() — проверка пустой корзины. Включает методы оформления заказа: clickPlaceOrder() — открытие модальной формы оформления, fillName(), fillCountry(), fillCity(), fillCard(), fillMonth(), fillYear() — заполнение полей формы, clickPurchase() — подтверждение заказа, closeCheckoutForm() — закрытие формы без оформления. Также реализованы проверочные методы: isCartTableVisible() — проверка отображения таблицы корзины, isPlaceOrderButtonVisible() — проверка наличия кнопки оформления, areHeadersVisible() — проверка заголовков таблицы, isPlaceOrderModalVisible() — проверка открытия модального окна.

ProductPage — класс, представляющий страницу отдельного товара. Содержит методы: clickAddToCart() — добавление товара в корзину, isTitleCorrect() — проверка названия товара, isPriceCorrect() — проверка цены, isDescriptionVisible() — проверка отображения описания, isAddToCartButtonVisible() — проверка наличия кнопки добавления. Для

работы с всплывающими уведомлениями реализованы методы: `acceptAlert()` — подтверждение всплывающего сообщения, `prepareAlertIntercept()` и `getCapturedAlertText()` — для перехвата и проверки текста системных сообщений (например, "Product added").

Пакет tests (тесты):

`BaseTest` — базовый класс для всех тестовых классов. Отвечает за жизненный цикл браузера: перед запуском каждого теста (`@BeforeEach`) настраивает браузер (размер окна 1920x1080, таймаут 10 секунд) и открывает главную страницу Demoblaze. После выполнения каждого теста (`@AfterEach`) закрывает браузер. Использует `LoggerUtil` для логирования начала и завершения тестов.

`Test01` — класс, содержащий тесты навигации по категориям: `testPhonesCategory()` — проверяет переход в категорию Phones и наличие товара Samsung Galaxy S6; `testLaptopsCategory()` — проверяет переход в категорию Laptops и наличие товара Sony Vaio i5.

`Test02` — тест открытия карточки товара `testOpenProductCard()`. Проверяет корректность отображения страницы товара Samsung Galaxy S6: названия, цены (\$360), описания и кнопки "Add to cart".

`Test03` — тест перехода в корзину `testCartNavigation()`. Проверяет, что при открытии корзины отображаются таблица с товарами (колонки Pic, Title, Price) и кнопка "Place Order".

`Test04` — класс, содержащий тесты добавления в корзину: `testAddToCartMessage()` — проверяет появление сообщения "Product added" после добавления товара; `testProductDisplayedInCart()` — проверяет, что добавленный товар отображается в корзине.

`Test05` — тест добавления нескольких товаров `testMultipleProductsInCart()`. Проверяет, что после последовательного добавления Samsung Galaxy S6 и Nokia Lumia 1520 в корзине отображаются оба товара.

Test06 — тест удаления товара из корзины `testDeleteProductFromCart()`. Проверяет, что после нажатия ссылки "Delete" товар Samsung Galaxy S6 исчезает из списка товаров в корзине.

Test07 — класс, содержащий тесты оформления заказа: `testOpenPlaceOrderForm()` — проверяет открытие модальной формы оформления заказа при нажатии кнопки "Place Order"; `testPurchaseWithCorrectData()` — проверяет успешное оформление заказа с заполнением всех полей формы и появление сообщения "Thank you for your purchase!".

Test08 — тест отмены заказа `testClosePlaceOrder()`. Проверяет, что при нажатии кнопки "Close" форма оформления заказа закрывается без выполнения покупки, и товар остается в корзине.

Test09 — класс, содержащий тесты пустой корзины: `testEmptyCart()` — проверяет, что в пустой корзине отсутствуют товары; `testProductEmptyCart()` — проверяет, что после добавления товара в пустую корзину он успешно отображается.

Пакет `utils` (вспомогательные утилиты):

`Constants` — класс-хранилище статических констант, используемых в тестах. Содержит названия товаров (S6, I5, NL), категорий (PH, LAP) и цену (PRICE). Использование констант упрощает поддержку тестов — при изменении тестовых данных достаточно внести правки в одном месте.

`LoggerUtil` — утилитный класс с полем `log` типа `java.util.logging.Logger`. Обеспечивает единообразное логирование в тестах. Используется в `BaseTest` для записи начала и завершения каждого теста, что помогает при отладке и анализе результатов выполнения.

2.2. UML-диаграмма классов

На рисунке (см. рис. 27) приведена UML-диаграмма классов, отражающая архитектуру разработанного тестового фреймворка. Диаграмма наглядно демонстрирует иерархию наследования между классами, а также

взаимосвязи между UI-элементами, страницами и тестовыми классами, что позволяет проследить логику взаимодействия компонентов в рамках паттерна Page Object Model.

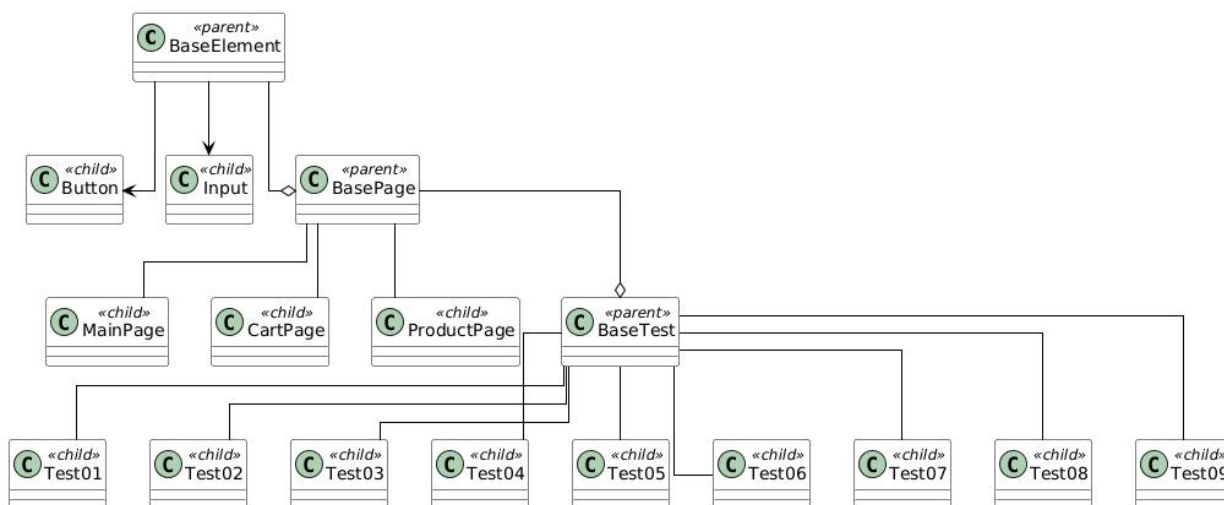


Рисунок 16 - UML-диаграмма

2.3. Реализация взаимодействия компонентов

Взаимодействие между классами построено по иерархическому принципу. Тестовые методы обращаются к методам классов страниц (Page Objects). Страницы, в свою очередь, делегируют управление специфическими блоками интерфейса элементам (Elements).

Например, при вызове метода `clickAddToCart()` в классе `ProductPage` происходит обращение к объекту `Button`, который выполняет нажатие на кнопку. При оформлении заказа методы `fillName()`, `fillCountry()` и другие из класса `CartPage` делегируют ввод данных объектам `Input`.

Такой подход позволяет изменять локаторы элементов в одном месте (в классе элемента) без необходимости править код тестов, что обеспечивает высокую поддерживаемость проекта.

3 ИНДИВИДУАЛЬНАЯ РАБОТА

В рамках работы над проектом мною была реализована логическая составляющая фреймворка, описывающая поведение страниц веб-приложения Demoblaze. Моя работа обеспечила возможность написания тестов, которые легко читаются и поддерживаются.

1. Проектирование и реализация Page Object Model (POM)

Разработка слоев страниц: спроектировал и реализовала классы MainPage, ProductPage и CartPage. Каждый класс был выстроен на основе принципа «одна страница — один класс», что позволило четко разграничить ответственность элементов и методов.

Инкапсуляция бизнес-логики: реализовал методы, описывающие полноценные сценарии взаимодействия пользователя с сайтом (навигация по категориям, добавление товаров в корзину, процесс оформления заказа, удаление товаров).

Интерфейсы взаимодействия: внедрение методов позволило скрыть внутреннюю структуру от самих тестов. Если селектор на сайте изменится, правки будут внесены только в один Page Object, не затрагивая десятки тестовых файлов.

2. Управление данными и конфигурацией (Constants)

Централизация данных: создание и наполнение класса Constants. Все «магические» значения (текстовые метки, названия категорий, цены) были вынесены в статические константы.

Поддерживаемость: такой подход исключил появление дубликатов в коде. При необходимости обновления тестовых данных (например, изменения имени товара или цены) правки производятся в одном месте, что минимизирует риск возникновения ошибок.

3. Обеспечение качества и читаемости кода

Унификация стилистики: провел работу по приведению всех Page Objects к единому стилю написания методов. Обеспечил именование методов

в соответствии с действиями, которые они выполняют, что сделало тесты самодокументируемыми.

Связность сценариев: реализовал логические связки между страницами, позволяя тестам переходить от выбора товара на MainPage к проверкам на ProductPage и завершать сценарий в CartPage.

Документирование: добавил Javadoc-комментарии к классам страниц, поясняющие назначение каждой сущности, что значительно упростило понимание структуры проекта для других участников команды.

4. Методологическая работа

Обработка состояний страниц: реализовал методы проверки видимости элементов (isProductVisible, isCartEmpty, isPurchaseSuccessVisible), которые позволяют тестам корректно верифицировать результат действий.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики были достигнуты поставленные цели и реализованы задачи по созданию системы автоматизированного UI-тестирования демонстрационного интернет-магазина Demoblaze. На первом этапе был проведен анализ основных функциональных возможностей сайта и выделены ключевые пользовательские сценарии, связанные с выбором категории товара, открытием карточки продукта, добавлением товаров в корзину, удалением товаров, оформлением заказа и отменой оформления. На основе проведенного анализа был подготовлен чек-лист, который охватывает основной путь пользователя от выбора товара до завершения покупки.

Вторая задача, связанная с разработкой архитектуры проекта, была решена с применением объектно-ориентированного подхода на языке Java и паттерна Page Object Model. В проекте были реализованы классы страниц MainPage, ProductPage и CartPage, которые отвечают за работу с главной страницей, карточкой товара и корзиной. Такой подход позволил отделить тестовые сценарии от логики взаимодействия с интерфейсом сайта и сделать структуру проекта более понятной и удобной для дальнейшего сопровождения.

Также в проекте были созданы базовые классы для работы со страницами и элементами интерфейса: BasePage, BaseElement, Button и Input. Они позволяют использовать общие методы для ожидания элементов, кликов по кнопкам, заполнения полей и проверки отображения элементов. Это помогло избежать дублирования кода и привести взаимодействие с элементами сайта к единому виду. Основные тестовые данные, такие как названия категорий, товаров и цена, были вынесены в класс Constants, что упростило поддержку тестов.

Третья задача, включающая программную реализацию автотестов и подготовку документации, была выполнена с использованием Selenide и JUnit 5. В проекте были реализованы тесты, проверяющие переход по категориям Phones и Laptops, открытие карточки товара Samsung Galaxy S6, переход в корзину, добавление одного и нескольких товаров, удаление товара, открытие

формы заказа, оформление покупки с корректными данными, отмену оформления заказа и проверку пустой корзины. Базовый класс `BaseTest` обеспечивает запуск каждого теста с открытием сайта `Demoblaze`, настройкой размера окна браузера, таймаута ожидания и закрытием браузера после выполнения проверки.

Для отслеживания выполнения тестов в проекте было реализовано базовое логирование с помощью класса `LoggerUtil`. Логирование используется при запуске и завершении тестов, а также при выполнении действий с элементами интерфейса. В отличие от исходного шаблона, в данном проекте не использовалась система `Log4j2`, так как логирование выполнено средствами стандартного Java-логгера.

В целом, работа над проектом позволила закрепить навыки проектирования структуры автоматизированных UI-тестов, применения паттерна `Page Object Model` и работы с инструментами `Selenide` и `JUnit 5`. Разработанная система автотестов проверяет основные функции интернет-магазина `Demoblaze` и может быть использована для повторной проверки стабильности работы пользовательских сценариев. Итоговые материалы проекта, включая исходный код, чек-лист, UML-диаграммы и отчетную документацию, были подготовлены для размещения в `GitHub`-репозитории и дальнейшего использования в рамках учебной практики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenium Documentation. Page object models [Электронный ресурс]. URL: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/ (дата обращения: 15.07.2026).
2. Selenide. Official documentation [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 15.07.2026).
3. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.07.2026).
4. Apache Maven. What is Maven? [Электронный ресурс]. URL: <https://maven.apache.org/what-is-maven.html> (дата обращения: 14.07.2024).
5. Oracle Java Documentation. Package java.util.logging. — URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.logging/java/util/logging/package-summary.html> (дата обращения: 14.07.2026).
6. <https://github.com/fl3xuz/Demoblaze-practice>